

Article

Long Short-Term Memory Neural Networks for Modeling Dynamical Processes and Predictive Control: A Hybrid Physics-Informed Approach

Krzysztof Zarzycki *  and Maciej Ławryńczuk 

Institute of Control and Computation Engineering, Faculty of Electronics and Information Technology, Warsaw University of Technology, ul. Nowowiejska 15/19, 00-665 Warsaw, Poland; maciej.lawrynczuk@pw.edu.pl

* Correspondence: krzysztof.zarzycki@pw.edu.pl

Abstract: This work has two objectives. Firstly, it describes a novel physics-informed hybrid neural network (PIHNN) model based on the long short-term memory (LSTM) neural network. The presented model structure combines the first-principle process description and data-driven neural sub-models using a specialized data fusion block that relies on fuzzy logic. The second objective of this work is to detail a computationally efficient model predictive control (MPC) algorithm that employs the PIHNN model. The validity of the presented modeling and MPC approaches is demonstrated for a simulated polymerization reactor. It is shown that the PIHNN structure gives very good modeling results, while the MPC controller results in excellent control quality.

Keywords: dynamical systems; LSTM neural networks; physics-informed neural networks; model predictive control



Citation: Zarzycki, K.; Ławryńczuk, M. Long Short-Term Memory Neural Networks for Modeling Dynamical Processes and Predictive Control: A Hybrid Physics-Informed Approach. *Sensors* **2023**, *23*, 8898. <https://doi.org/10.3390/s23218898>

Academic Editors: Ching-Hung Lee and Lian-Wang Lee

Received: 4 October 2023

Revised: 26 October 2023

Accepted: 30 October 2023

Published: 1 November 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Model predictive control (MPC) algorithms, as highlighted in [1,2], find their primary applications in managing processes that classical control methods struggle to handle effectively. These processes often involve multiple-input, multiple-output (MIMO) systems or exhibit strong nonlinearity. MPC, renowned for its flexibility in accommodating various constraints, excels in ensuring high-quality control, even in the face of challenging processes. Real-world instances of successful MPC applications include control of chemical reactors [3,4] and distillation towers [5], as well as the integration of MPC in embedded systems controlling heating ventilation and air conditioning systems (HVAC) [6], quadrotors [7], fuel cells [8], autonomous vehicles [9], and underwater vehicles [10].

As emphasized in [11–13], accurate sensor measurements of essential process variables play a critical role in MPC. It is widely acknowledged that the absence of these measurements inevitably leads to a significant loss in control performance. To address this challenge, when the necessary measurements are not readily available, engineers commonly employ online estimation techniques, such as Kalman or extended Kalman filters ([14]). Furthermore, specialized methods and strategies have been developed to tackle this issue in specific applications. In the domain of vehicles, innovative solutions have emerged. The authors of [15] introduce a real-world example where a vehicle employs an external camera to detect obstacles and lane positions on the road. Additionally, it utilizes external rear-corner radars to identify objects approaching from the rear. An intriguing application of sensors is presented in [16], where an anemometer measures external factors such as wind force and direction. Beyond the automotive sector, there are applications like sea ship depth measurement. In [10], a depth sensor is installed for precisely measuring sea ship depth, with heave speed derived from the depth sensor data. Finally, MPC is also used to manage fault-tolerant control. This application addresses issues like stiction in control valves, as discussed in [17].

The cornerstone of any effective MPC algorithm is the precision of its process model. Broadly, two general model classes are usually considered: first-principle (FP) models rooted in the fundamental understanding of the process; black-box approximations. Both model classes have their distinct strengths and limitations:

- FP models demand meticulous process descriptions and accurate parameter values but offer unparalleled modeling precision across a wide operating range, even in abnormal situations. In practice, however, the values of some model parameters may be imprecise or unknown.
- In contrast, data-driven black-box models, including support-vector machines (SVM) [18], multi-layer perceptron (MLP) neural networks [19,20], radial basis function (RBF) networks [21], and recurrent long short-term memory networks (LSTM) [22,23], require no prior domain expertise. Among these, gated recurrent unit (GRU) has gained traction in modeling dynamic systems [24,25] and integrating with MPC algorithms [23,26,27]. Neural networks models have proven to be very useful, especially when dealing with complex dynamical processes, such as predator–prey systems [28–30]. However, black-box models may struggle when the available dataset lacks coverage for certain process variables, particularly those operating at infrequent points.

Physics-informed neural network models (PINNs) offer a compelling fusion of both modeling approaches. These models combine the foundational principles governing the process with the data-driven power of machine learning. The result is a versatile model that adheres to fundamental laws while approximating the behavior of real-world processes. The literature showcases PINN applications in scenarios where parameters of ordinary differential equations (ODE) models are either imprecisely known [31] or immeasurable [32]. Furthermore, PINNs can approximate parameters of partial differential equations (PDE) [33]. These PINN models find utility in replacing numerical solvers for ODEs [34] and even serve as models within MPC frameworks [35]. Additionally, one can find several hybrid models aiming to combine a data-driven modeling approach with knowledge of physics. The hybrid physical guided neural network [36] is a feed-forward neural network integrated with a first-principles model. The entire hybrid model is trained jointly. This training process involves incorporating a fusion output layer that utilizes a straightforward interpolation technique. Other examples include using deep neural networks in a physically guided modeling approach [37], in modeling lithium batteries [38], and in modeling a traffic state [39]. One can also find examples of introducing physics directly in the forward pass of the neural network to model the lake temperature [40].

This study addresses a common modeling challenge characterized by two specific limitations. Firstly, process-variable measurements are typically feasible but confined to a limited vicinity of certain operating points. Consequently, the resulting models exhibit localized validity, restricted to the regions where data have been collected for identification purposes. Secondly, although fundamentally sound, the existing first-principle models describing the process often lack precision due to imprecise parameters. In response to these limitations, this work introduces an innovative physics-informed hybrid neural network (PIHNN) model structure, leveraging LSTM neural networks. This approach combines elements from both first-principle and black-box data-driven methodologies, offering robust modeling capabilities in scenarios characterized by the aforementioned issues. Within this research, we delve into two data fusion techniques, drawing from the principles of the first-principle process description and the LSTM network, both employing a fuzzy-logic-based approach. The initial method employs a simplified data fusion block, while the subsequent method harnesses machine learning techniques to minimize overall model errors. To assess the effectiveness of the proposed model structure and data fusion techniques, we apply them to a benchmark polymerization reactor process.

Additionally, we integrate the developed PIHNN model into the MPC framework. Our analysis encompasses a straightforward MPC algorithm with a nonlinear optimization MPC (MPC-NO) and a more intricate linearization-based MPC scheme named the MPC algorithm with nonlinear prediction and linearization around the predicted trajectory (MPC-NPLPT),

which relies on computationally uncomplicated quadratic optimization tasks. Our findings demonstrate that the linearization-based MPC approach can yield commendable control performance while significantly reducing computational demands compared to nonlinear counterparts. An initial iteration of the PIHNN model was introduced in conference proceedings [41], where a basic GRU neural network was employed. This current study represents a substantial expansion of previous research efforts. Here, we consider more general LSTM-based PIHNN models, comprehensively examine the model's structure, explore various potential variants and present details. Furthermore, we introduce an efficient model predictive control (MPC) algorithm for the PIHNN models considered in this study.

This work is organized as follows. Firstly, Section 2 presents the general structure and the details of the hybrid PIHNN model structure utilizing LSTM neural networks. The state-space modeling approach is employed. Secondly, Section 3 briefly describes the general MPC scheme with nonlinear optimization and presents general formulation, necessary implementation details, and the resulting quadratic optimization task of the linearization-based MPC method. Section 4 thoroughly studies the validity of as many as six PIHNN model variants applied to approximate the behavior of a chemical reactor benchmark. Furthermore, the control efficiency and computational speed of the recommenced linearization-based MPC algorithm is shown. Finally, Section 5 concludes the article.

2. Hybrid Physics-Informed Models Using LSTM Neural Networks

We introduce an innovative PIHNN model that blends a data-driven approach with expert knowledge of the underlying physics of the process. To effectively apply the PIHNN model, the following conditions must be satisfied:

- The process input and output variables, i.e., the manipulated and controlled variables, respectively, must be measurable. State variables may be measured or observed using a state estimation, e.g., in the form of an extended Kalman filter (EKF).
- The FP process should exist in the form of a set of differential equations and, when necessary, additional algebraic relations based on the fundamental laws of physics governing the process.

However, we assume that the measurements and the FP model may exhibit imperfections. Specifically, the measurements may originate from a limited range within the entire spectrum of process variable variability. Furthermore, the FP model may also contain inaccuracies and be susceptible to errors arising from factors such as incorrect estimation of specific process parameters or measurement inaccuracies.

2.1. Model Structure

This paper primarily focuses on single-input single-output (SISO) process modeling. The process input and output are denoted as u and y , respectively. Additionally, the process has n_x state variables, represented as $x = [x_1 \dots x_{n_x}]^T$.

Figure 1 illustrates the model's overall structure. The PIHNN model is divided into three distinct components. The first model component, highlighted in blue, is entirely data-driven. It comprises n_{LSTM} neural sub-models, each trained on available data. The number of data-driven sub-models corresponds to the number of distinct operational areas of the process from which measurement data can be collected. Each sub-model takes the vector $\mathbf{X}_{\text{LSTM}}^i$ as the input and generates the scalar y_{LSTM}^i as the output. LSTM networks are employed in this study, as earlier research has demonstrated their exceptional ability to model dynamical processes [23,27]. However, it is important to note that alternative data-driven models could also be applied in this context. The second component of the PIHNN structure, highlighted in green, is rooted in expert knowledge about the underlying physics of the process. It consists of an FP sub-model formulated using ordinary differential equations. The input to this sub-model is the vector $\mathbf{X}_{\text{FP}}$, while the output is denoted by the scalar y_{FP} . The third component of the PIHNN structure, highlighted in orange, represents the data fusion block (DF). In general, many decision models can be used here, such as

neural networks of various architectures. However, we recommend using the fuzzy data fusion block (Fuzzy DF) because it directly incorporates the sub-models. There is no need to train Fuzzy DF on data, which is particularly useful when training data is lacking across specific ranges of process variable variability. By selecting membership function shapes, one can determine which areas and to what extent we should consider the sub-models when calculating the overall PIHNN model output. The DF block takes output calculated by all LSTM sub-models and the FP models as inputs. Based on the current operating state of the process, represented by the vector $\mathbf{X}_{DF}$, it makes decisions regarding the combination of outputs from all sub-models. The primary goal of this fusion process is to minimize the overall error of the entire PIHNN model.

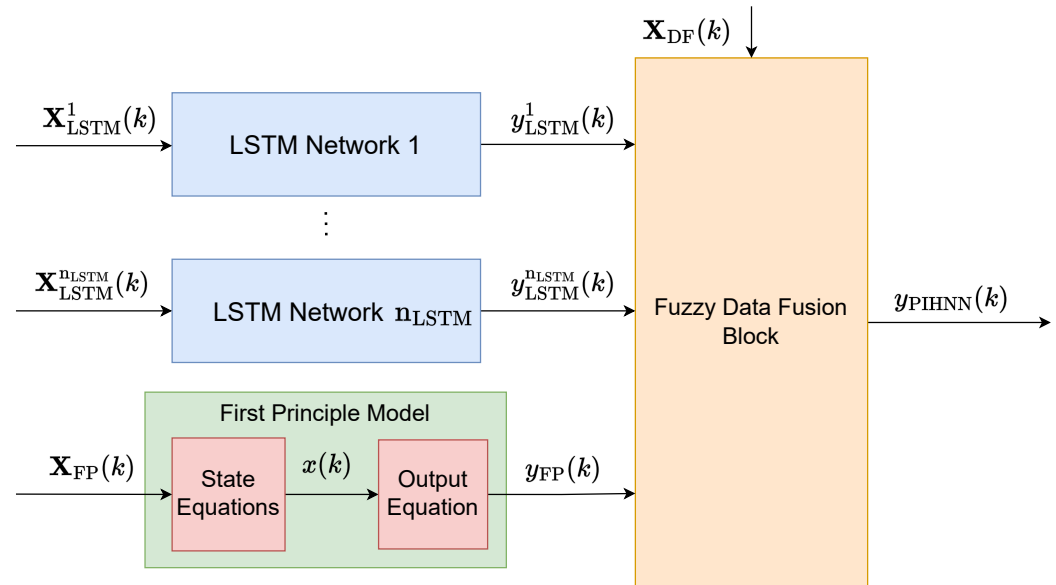


Figure 1. General structure of the PIHNN model.

2.2. First-Principle Sub-Model

Typically, the FP model utilizes fundamental physical laws formulated in the continuous-time domain, i.e., a set of differential equations must be considered. The state equations have the classical form

$$\dot{x}_1(t) = f_1(x_1(t), \dots, x_{n_x}(t), u(t)) \quad (1)$$

$$\vdots$$

$$\dot{x}_{n_x}(t) = f_{n_x}(x_1(t), \dots, x_{n_x}(t), u(t)) \quad (2)$$

while the output equation is

$$y(t) = g(x_1(t), \dots, x_{n_x}(t)) \quad (3)$$

where $f_1, \dots, f_{n_x} : \mathbf{R}^{n_x+1} \rightarrow \mathbf{R}$ and $g : \mathbf{R} \rightarrow \mathbf{R}$ are nonlinear functions. Since we will next use the PIHNN model relying on the FP model in the MPC algorithm with online linearization, we require the functions $f_1, \dots, f_{n_x}, g$ to be differentiable. From Equations (1)–(3), we can find a corresponding discrete-time FP model

$$x_1(k) = f_1^d(x_1(k-1), \dots, x_{n_x}(k-1), u(k-1)) \quad (4)$$

$$\vdots$$

$$x_{n_x}(k) = f_{n_x}^d(x_1(k-1), \dots, x_{n_x}(k-1), u(k-1)) \quad (5)$$

$$y(k) = g^d(x_1(k-1), \dots, x_{n_x}(k-1)) \quad (6)$$

where $f_1^d, \dots, f_{n_x}^d : \mathbf{R}^{n_x+1} \rightarrow \mathbf{R}$ and $g^d : \mathbf{R} \rightarrow \mathbf{R}$ are nonlinear mapping functions. The input vector to the FP model can be expressed as

$$\mathbf{X}_{\text{FP}}(k) = \begin{bmatrix} x^T(k-1) & u(k-1) \end{bmatrix}^T \quad (7)$$

2.3. LSTM Sub-Model

LSTM networks were developed in response to the vanishing gradient problem that impacts traditional recurrent neural networks [42]. Each LSTM neuron is referred to as a “cell” (Figure 2) and encompasses gates responsible for governing the flow of information within the network. The LSTM cell comprises four distinct gates:

- The forget gate f determines which values from the previous cell state should be retained and which should be discarded.
- The input gate i selects values from both the previous hidden state and the current input for updating purposes.
- The cell state candidate gate g initially regulates the flow of information within the network and subsequently computes the candidate value for the current cell state.
- The output gate o is responsible for calculating the new hidden state ‘ h ’.

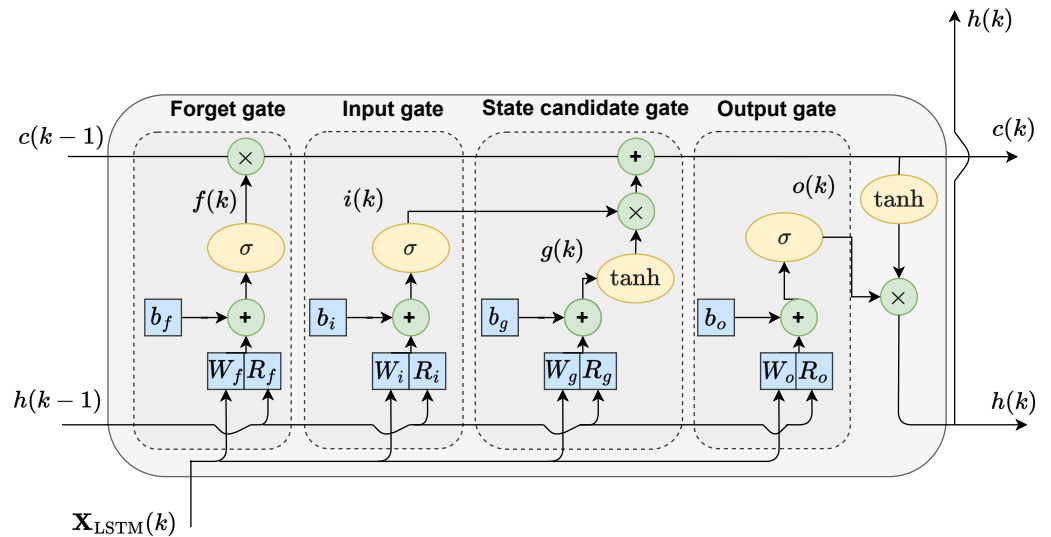


Figure 2. Structure of the LSTM cell.

Each cell in the network has its input vector expressed as

$$\mathbf{X}_{\text{LSTM}}(k) = [x_{\text{LSTM}}^1, \dots, x_{\text{LSTM}}^m]^T = [u(k-1), \dots, u(k-n_B), y(k-1), \dots, y(k-n_A)]^T \quad (8)$$

where parameters n_A and n_B define the order of dynamics of the model. The LSTM network has n_N cells. The weights in the network can be written in a matrix form

$$\mathbf{W} = \begin{bmatrix} W_i \\ W_f \\ W_g \\ W_o \end{bmatrix}, \mathbf{R} = \begin{bmatrix} R_i \\ R_f \\ R_g \\ R_o \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_i \\ b_f \\ b_g \\ b_o \end{bmatrix} \quad (9)$$

The input weight matrices W_i , W_f , W_g and W_o have dimensionality $n_N \times n_f$; the recurrent weight matrices R_i , R_f , R_g and R_o have dimensionality $n_N \times n_N$; and the bias vectors b_i ,

$\mathbf{b}_f$, $\mathbf{b}_g$ and $\mathbf{b}_o$ have dimensionality $n_N \times 1$, respectively. At time instant k , the LSTM model initially calculates the output value of each gate

$$\mathbf{i}(k) = \sigma \left(\mathbf{W}_i \mathbf{X}_{\text{LSTM}} + \mathbf{R}_i \mathbf{h}(k-1) + \mathbf{b}_i \right) \quad (10)$$

$$\mathbf{f}(k) = \sigma \left(\mathbf{W}_f \mathbf{X}_{\text{LSTM}} + \mathbf{R}_f \mathbf{h}(k-1) + \mathbf{b}_f \right) \quad (11)$$

$$\mathbf{g}(k) = \tanh \left(\mathbf{W}_g \mathbf{X}_{\text{LSTM}} + \mathbf{R}_g \mathbf{h}(k-1) + \mathbf{b}_g \right) \quad (12)$$

$$\mathbf{o}(k) = \sigma \left(\mathbf{W}_o \mathbf{X}_{\text{LSTM}} + \mathbf{R}_o \mathbf{h}(k-1) + \mathbf{b}_o \right) \quad (13)$$

Subsequently, the cell state of the network can be computed

$$\mathbf{c}(k) = \mathbf{f}(k) \circ \mathbf{c}(k-1) + \mathbf{i}(k) \circ \mathbf{g}(k) \quad (14)$$

where the symbol $\circ$ represents the Hadamard product of vectors. Finally, the hidden state can be calculated

$$\mathbf{h}(k) = \mathbf{o}(k) \circ \tanh(\mathbf{c}(k)) \quad (15)$$

The LSTM layer of the network is typically added to a fully connected layer (Figure 3), with weight matrix $\mathbf{W}_y$ with a dimensionality of $1 \times n_N$ and bias $\mathbf{b}_y$. Finally, the computation of the network's output at time instant k can be expressed as

$$\mathbf{y}_{(i)}^{\text{LSTM}}(k) = \mathbf{W}_y \mathbf{h}(k) + \mathbf{b}_y \quad (16)$$

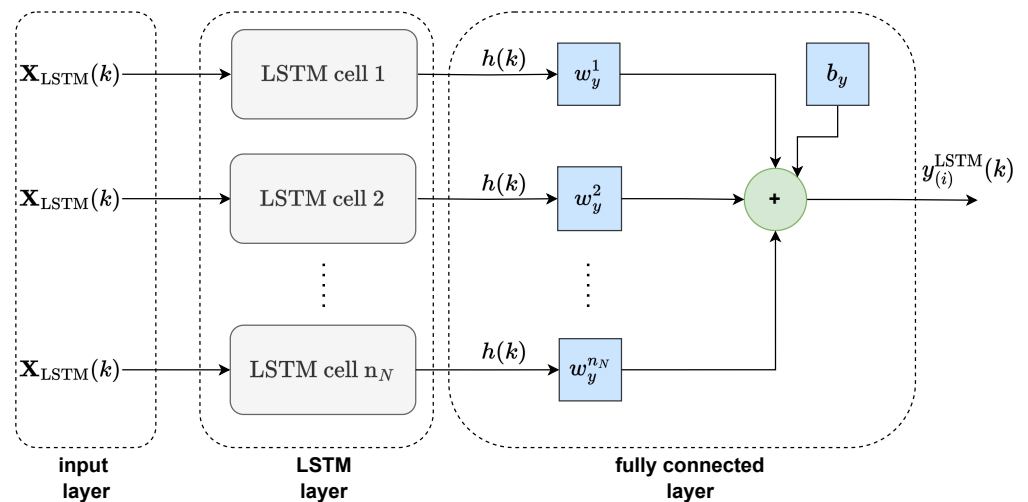


Figure 3. Structure of the whole LSTM network.

One can represent Equations (10)–(15) in scalar form, which will prove useful for the derivation of the MPC algorithm considered in Section 3. The scalar form expressions for the n -th elements of the gate and state vectors are

$$i_n(k) = \sigma \left(\sum_{m=1}^{n_A+n_B} (w_{n,m}^i x_{\text{LSTM}}^m(k)) + \sum_{m=1}^{n_N} (r_{n,m}^i h_m(k-1)) + b_n^i \right) \quad (17)$$

$$f_n(k) = \sigma \left(\sum_{m=1}^{n_A+n_B} (w_{n,m}^f x_{\text{LSTM}}^m(k)) + \sum_{m=1}^{n_N} (r_{n,m}^f h_m(k-1)) + b_n^f \right) \quad (18)$$

$$g_n(k) = \tanh \left(\sum_{m=1}^{n_A+n_B} (w_{n,m}^g x_{LSTM}^m(k)) + \sum_{m=1}^{n_N} (r_{n,m}^g h_m(k-1)) + b_n^g \right) \quad (19)$$

$$o_n(k) = \sigma \left(\sum_{m=1}^{n_A+n_B} (w_{n,m}^o x_{LSTM}^m(k)) + \sum_{m=1}^{n_N} (r_{n,m}^o h_m(k-1)) + b_n^o \right) \quad (20)$$

$$c_n(k) = f_n(k) c_n(k-1) + i_n(k) g_n(k) \quad (21)$$

$$h_n(k) = o_n(k) \tanh(c_n(k)) \quad (22)$$

$$y_{(i)}^{LSTM}(k) = \sum_{n=1}^{n_N} w_y^n h_n(k) + b_y \quad (23)$$

Equations (22) and (23) could be used to find the output of the network in the form of one equation

$$y_{(i)}^{LSTM}(k) = \sum_{n=1}^{n_N} w_y^n \left(o_n(k) \tanh(f_n(k) c_n(k-1) + i_n(k) g_n(k)) \right) + b_y \quad (24)$$

2.4. Fuzzy Data Fusion Block

Considering Figure 1, the output of the whole PIHNN model is

$$y_{PIHNN}(k) = \frac{\sum_{n=1}^{n_{LSTM}} y_n^{LSTM}(k) \mu_n^{LSTM}(k) + y_{FP}(k) \left(\sum_{n=1}^{n_{FP}} \mu_n^{FP}(k) \right)}{\sum_{n=1}^{n_{LSTM}} \mu_n^{LSTM}(k) + \sum_{n=1}^{n_{FP}} \mu_n^{FP}(k)} \quad (25)$$

In this study, we use trapezoidal, sigmoidal, and Gaussian membership functions. For trapezoidal functions, we have

$$\mu_n^{LSTM}(k) = \mu^{LSTM}(\mathbf{X}_{DF}(k)) = \max \left(\min \left(\frac{\mathbf{X}_{DF}(k) - a_n}{b_n - a_n}, 1, \frac{d_n - \mathbf{X}_{DF}(k)}{d_n - c_n} \right), 0 \right) \quad (26)$$

for sigmoidal ones, we write

$$\mu_n^{LSTM}(k) = \mu^{LSTM}(\mathbf{X}_{DF}(k)) = \frac{1}{1 + e^{-a_n(\mathbf{X}_{DF}(k) - b_n)}} \frac{1}{1 + e^{-c_n(\mathbf{X}_{DF}(k) - d_n)}} \quad (27)$$

and for Gaussian ones, we define

$$\mu_n^{LSTM}(k) = \mu^{LSTM}(\mathbf{X}_{DF}(k)) = \exp \left(\frac{-(\mathbf{X}_{DF}(k) - a_n)}{2b_n^2} \right) \quad (28)$$

The signal $\mathbf{X}_{DF}(k) = y(k)$ or $\mathbf{X}_{DF}(k) = u(k-1)$ defines the current operating point of the process. Parameters a_n, b_n, c_n, d_n define the shape of membership functions used.

2.5. Model Development Procedure

The process for establishing the PIHNN model unfolds as follows:

1. We determine the number of distinct training datasets that can be derived from the process measurements.
2. We conduct training of the LSTM network for each training dataset.
3. We implement a discrete FP model of the process.
4. We select the initial shape and range of the membership function within the DF block.
5. We deliver the outputs of the LSTM sub-models and the output of the FP model as inputs of the DF block, where their fusion is carried out based on the current operational state of the process. This fusion process determines the output of the PIHNN model.

6. We assess the quality of PIHNN modeling. If it proves unsatisfactory, then it becomes necessary to modify the shape of the membership function.
7. We adjust the membership function's shape, which can be executed manually, drawing upon expert knowledge, or using an optimization procedure.

The flow chart of the model development procedure is also presented in Figure 4.

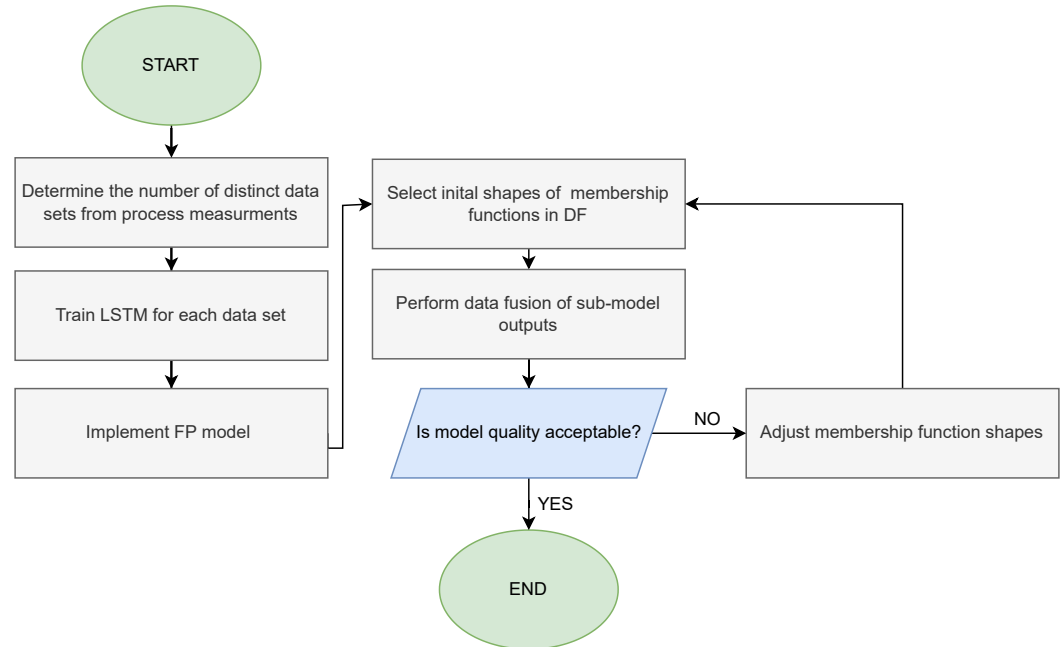


Figure 4. Flow chart for development of PIHNN model.

3. LSTM PIHNN Models in Predictive Control

3.1. Basic Predictive Control Problem Formulation

This work utilizes the general MPC formulation [1,2]. Namely, at each discrete-time sampling instant k , where $k = 0, 1, 2, \dots$, the MPC controller performs real-time calculations to determine the vector of decision variables. It is defined as the following current and future increments of the input variable

$$\Delta \mathbf{u}(k) = [\Delta u(k|k) \ \Delta u(k+1|k) \ \dots \ \Delta u(k+N_u-1|k)]^T \quad (29)$$

The symbol $\Delta u(k|k)$ represents the increment of the manipulated variable at time instant k , computed at the same time instant k . Similarly, the symbol $\Delta u(k+1|k)$ corresponds to the increment of the manipulated variable at the future time instant $k+1$, computed at the current time instant k . This notation extends to subsequent time instants as well. N_u represents the control horizon, which determines the length of the MPC decision variable vector. The fundamental MPC optimization problem aims to minimize the predicted control error, minimize excessive increments of the manipulated variable, and satisfy constraints. Let us denote the set-point of the controlled variable for the future sampling instant $k+p$ known at the current instant k by $y^{sp}(k+p|k)$ and the corresponding prediction determined from the process model by $\hat{y}(k+p|k)$. We consider the predictions and control errors on the prediction horizon N . As far as the magnitude constraints on the manipulated variable and the predicted controlled variable are concerned, they are represented by $u^{\min}$, $u^{\max}$

and $y^{\min}$, $y^{\max}$, respectively. The fundamental MPC optimization task can be formulated as follows:

$$\begin{aligned} \min_{\Delta u(k)} \left\{ J(k) = \sum_{p=1}^N (y^{\text{sp}}(k+p|k)^2 - \hat{y}(k+p|k)) + \lambda \sum_{p=0}^{N_u-1} (\Delta u(k+p|k))^2 \right\} \\ \text{subject to} \\ u^{\min} \leq u(k+p|k) \leq u^{\max}, \quad p = 0, \dots, N_u - 1 \\ \Delta u^{\min} \leq \Delta u(k+p|k) \leq \Delta u^{\max}, \quad p = 0, \dots, N_u - 1 \\ y^{\min} \leq \hat{y}(k+p|k) \leq y^{\max}, \quad p = 1, \dots, N. \end{aligned} \quad (30)$$

In general, the predictions over the prediction horizon are obtained as

$$\hat{y}(k+p|k) = y(k+p|k) + d(k) \quad (31)$$

where the model output for the future discrete time $k+p$, determined at the current time k , is denoted as $y(k+p|k)$. The unmeasured disturbance, that covers the model error and real disturbances that act on the controlled process, is computed as the difference between the measured value of the process controlled variable and its estimation obtained from the model. The MPC optimization problem (30) is solved online at each sampling instant, yielding the solution vector (29). According to the principle of repetitive control, the first element of the obtained solution vector is sent to the process and the whole procedure is repeated in the subsequent sampling instants.

3.2. Nonlinear MPC Optimization for PIHNN Models

Suppose a nonlinear model, e.g., an LSTM structure or the PIHNN model described in this work, is directly used to determine the predictions $\hat{y}(k+p|k)$. The general MPC optimization problem (30) becomes nonlinear in that case. We will refer to such a control method as MPC-NO.

3.3. Quadratic MPC Optimization for PIHNN Models

In order to derive a computationally attractive alternative to the MPC-NO method, we derive an MPC with successive linearization of the predicted trajectory. Such an approach will make it possible to derive a quadratic optimization MPC task. We use the general approach to predicted trajectory linearization known as the MPC-NPLPT method, introduced in [19,20]. However, the application of an original PIHNN model structure requires careful derivation of the algorithm. Firstly, let us define the predicted trajectory of the controlled variable over the entire prediction horizon, i.e., the following vector:

$$\hat{\mathbf{y}}(k) = [\hat{y}(k+1|k) \dots \hat{y}(k+N|k)]^T \quad (32)$$

In the MPC-NPLPT approach, linearization is performed along a trajectory of the manipulated variable defined over the control horizon. It has the following form:

$$\mathbf{u}^{\text{traj}}(k) = [u^{\text{traj}}(k|k) \dots u^{\text{traj}}(k+N_u-1|k)]^T \quad (33)$$

From the definition of the control horizon, it follows that $u^{\text{traj}}(k+p|k) = u^{\text{traj}}(k+N_u-1|k)$ for $p = N_u, \dots, N$. The input trajectory (33) is utilized to determine the predicted trajectory of the controlled variable over the prediction horizon

$$\hat{\mathbf{y}}^{\text{traj}}(k) = [\hat{y}^{\text{traj}}(k+1|k) \dots \hat{y}^{\text{traj}}(k+N|k)]^T \quad (34)$$

For linearization, we use Taylor's approach. Let us define the vector comprising the current and future values of the manipulated variable that correspond to the MPC decision variable vector (29)

$$\mathbf{u}(k) = [u(k|k) \dots u(k + N_u - 1|k)]^T \quad (35)$$

Taking advantage of the compact vector–matrix notation, the predicted trajectory, $\hat{\mathbf{y}}(k)$, is expressed as the following linear function of the vector (35):

$$\hat{\mathbf{y}}(k) = \hat{\mathbf{y}}^{\text{traj}}(k) + \mathbf{H}(k)(\mathbf{u}(k) - \mathbf{u}^{\text{traj}}(k)) \quad (36)$$

The $N \times N_u$ matrix

$$\mathbf{H}(k) = \frac{d\hat{\mathbf{y}}^{\text{traj}}(k)}{d\mathbf{u}^{\text{traj}}(k)} \quad (37)$$

defines partial derivatives of the predicted controlled variable's trajectory with respect to the future manipulated variable's trajectory; both trajectories take into account the linearization conditions, so we have to utilize the trajectories $\hat{\mathbf{y}}^{\text{traj}}(k)$ and $\mathbf{u}^{\text{traj}}(k)$, respectively. The entries of the matrix $\mathbf{H}(k)$ are

$$H_{r+1,p}(k) = \frac{\partial \hat{y}^{\text{traj}}(k+p|k)}{\partial u^{\text{traj}}(k+r|k)} \quad (38)$$

for all predictions over the prediction horizon, i.e., $p = 1, \dots, N$, and all computed values of the manipulated variable over the entire control horizon, i.e., $r = 0, \dots, N_u$. The link between the vectors $\mathbf{u}(k)$ and $\Delta \mathbf{u}(k)$ is

$$\mathbf{u}(k) = \mathbf{J}\Delta \mathbf{u}(k) + \mathbf{u}(k-1) \quad (39)$$

when the entries of the $N_u \times N_u$ auxiliary matrix $\mathbf{J}$ are defined as

$$J_{i,j} = \begin{cases} 0 & \text{if } i < j \\ 1 & \text{if } i \geq j \end{cases} \quad (40)$$

and the vector of length N_u is

$$\mathbf{u}(k-1) = [u(k-1) \dots u(k-1)]^T \quad (41)$$

Using the linearized trajectory (36) and the rule (39), the general predictive control optimization task (30) is transformed to the subsequent quadratic optimization problem, as follows:

$$\begin{aligned} & \min_{\Delta \mathbf{u}(k)} \left\{ \|\mathbf{y}^{\text{sp}}(k) - \mathbf{H}(k)\mathbf{J}\Delta \mathbf{u}(k) - \hat{\mathbf{y}}(k) - \mathbf{H}(k)(\mathbf{u}(k-1) - \mathbf{u}(k))\|^2 + \|\Delta \mathbf{u}(k)\|_{\Lambda}^2 \right\} \\ & \text{subject to} \\ & \mathbf{u}^{\min} \leq \mathbf{J}\Delta \mathbf{u}(k) + \mathbf{u}(k-1) \leq \mathbf{u}^{\max} \\ & \Delta \mathbf{u}^{\min} \leq \Delta \mathbf{u}(k) \leq \Delta \mathbf{u}^{\max} \\ & \mathbf{y}^{\min} \leq \mathbf{H}(k)\mathbf{J}\Delta \mathbf{u}(k) + \hat{\mathbf{y}}(k) + \mathbf{H}(k)(\mathbf{u}(k-1) - \mathbf{u}(k)) \leq \mathbf{y}^{\max} \end{aligned} \quad (42)$$

The definitions for all necessary symbols used in the above problem are

- Λ : a diagonal $N_u \times N_u$ matrix with diagonal entries equal to the weighting coefficient λ ;
- $\mathbf{u}^{\min}$: a vector of length N_u , where all elements are equal to $u^{\min}$;
- $\mathbf{u}^{\max}$: a vector of length N_u , where all elements are equal to $u^{\max}$;
- $\Delta \mathbf{u}^{\min}$: a vector of length N_u , where all elements are equal to $\Delta u^{\min}$;
- $\Delta \mathbf{u}^{\max}$: a vector of length N_u , where all elements are equal to $\Delta u^{\max}$;
- $\mathbf{y}^{\min}$: a vector of length N , where all elements are equal to $y^{\min}$;
- $\mathbf{y}^{\max}$: a vector of length N , where all elements are equal to $y^{\max}$.

3.4. PIHNN Prediction

Let us now discuss how the PIHNN model discussed in this work is utilized for MPC prediction, i.e., to calculate the predicted trajectory of the controlled variable defined by Equation (34). We use Equation (25) for the future time instant $k + p$ which gives

$$y^{\text{PIHNN}}(k + p|k) = \frac{\sum_{n=1}^{n_{\text{LSTM}}} y_n^{\text{LSTM}}(k + p|k) \mu_n^{\text{LSTM}}(k) + y_{\text{FP}}(k + p|k) \left(\sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k) \right)}{\sum_{n=1}^{n_{\text{LSTM}}} \mu_n^{\text{LSTM}}(k) + \sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k)} \quad (43)$$

Taking advantage of Equation (31), the predictions are, therefore, expressed as

$$\hat{y}^{\text{PIHNN}}(k + p|k) = \frac{\sum_{n=1}^{n_{\text{LSTM}}} y_n^{\text{LSTM}}(k + p|k) \mu_n^{\text{LSTM}}(k) + y_{\text{FP}}(k + p|k) \left(\sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k) \right)}{\sum_{n=1}^{n_{\text{LSTM}}} \mu_n^{\text{LSTM}}(k) + \sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k)} + d(k) \quad (44)$$

where the membership functions are defined by Equations (26), (27) or (28). Let us note that the predicted trajectory from the PIHNN model depends on the trajectories generated by both LSTM and FP sub-models. The disturbance (the prediction error) is determined as the difference between the measured process output and its estimation obtained from the model

$$d(k) = y(k) - \hat{y}^{\text{PIHNN}}(k) \quad (45)$$

where the signal $\hat{y}^{\text{PIHNN}}(k)$ is found from Equation (31).

3.5. LSTM Model Prediction

For each LSTM sub-model, the calculations start with computing the predicted output of the gates. For this purpose, we use Equations (17)–(20) which yield the following

$$i_n(k + p|k) = \sigma \left(\sum_{m=1}^{n_A + n_B} (w_{n,m}^i x_{\text{LSTM}}^m(k + p|k)) + \sum_{m=1}^{n_N} (r_{n,m}^i h_m(k - 1 + p|k)) + b_n^i \right) \quad (46)$$

$$f_n(k + p|k) = \sigma \left(\sum_{m=1}^{n_A + n_B} (w_{n,m}^f x_{\text{LSTM}}^m(k + p|k)) + \sum_{m=1}^{n_N} (r_{n,m}^f h_m(k - 1 + p|k)) + b_n^f \right) \quad (47)$$

$$g_n(k + p|k) = \tanh \left(\sum_{m=1}^{n_A + n_B} (w_{n,m}^g x_{\text{LSTM}}^m(k + p|k)) + \sum_{m=1}^{n_N} (r_{n,m}^g h_m(k - 1 + p|k)) + b_n^g \right) \quad (48)$$

$$o_n(k + p|k) = \sigma \left(\sum_{m=1}^{n_A + n_B} (w_{n,m}^o x_{\text{LSTM}}^m(k + p|k)) + \sum_{m=1}^{n_N} (r_{n,m}^o h_m(k - 1 + p|k)) + b_n^o \right) \quad (49)$$

Let us introduce auxiliary integer variables as follows: $I_{\text{uf}}(p) = \max(\min(p, n_B), 0)$, $I_{\text{yf}}(p) = \min(p - 1, n_A)$. We can represent gate predictions as

$$\begin{aligned}
i_n(k+p|k) = & \sigma \left(\sum_{m=1}^{I_{uf}(p)} w_{n,m}^i u(k-m+p|k) + \sum_{m=I_{uf}(p)+1}^{n_B} w_{n,I_{uf}(p)+m}^i u(k-m+p) \right. \\
& + \sum_{m=1}^{I_{yf}(p)} w_{n,n_B+m}^i \hat{y}^{LSTM}(k-m+p|k) + \sum_{m=I_{yf}(p)+1}^{n_A} w_{n,I_{yf}(p)+m}^i y(k-m+p) \\
& \left. + \sum_{m=1}^{n_N} r_{n,m}^i h_m(k+p-1|k) + b_n^i \right)
\end{aligned} \quad (50)$$

$$\begin{aligned}
f_n(k+p|k) = & \sigma \left(\sum_{m=1}^{I_{uf}(p)} w_{n,m}^f u(k-m+p|k) + \sum_{m=I_{uf}(p)+1}^{n_B} w_{n,I_{uf}(p)+m}^f u(k-m+p) \right. \\
& + \sum_{m=1}^{I_{yf}(p)} w_{n,n_B+m}^f \hat{y}^{LSTM}(k-m+p|k) + \sum_{m=I_{yf}(p)+1}^{n_A} w_{n,I_{yf}(p)+m}^f y(k-m+p) \\
& \left. + \sum_{m=1}^{n_N} r_{n,m}^f h_m(k+p-1|k) + b_n^f \right)
\end{aligned} \quad (51)$$

$$\begin{aligned}
g_n(k+p|k) = & \tanh \left(\sum_{m=1}^{I_{uf}(p)} w_{n,m}^g u(k-m+p|k) + \sum_{m=I_{uf}(p)+1}^{n_B} w_{n,I_{uf}(p)+m}^g u(k-m+p) \right. \\
& + \sum_{m=1}^{I_{yf}(p)} w_{n,n_B+m}^g \hat{y}^{LSTM}(k-m+p|k) + \sum_{m=I_{yf}(p)+1}^{n_A} w_{n,I_{yf}(p)+m}^g y(k-m+p) \\
& \left. + \sum_{m=1}^{n_N} (r_{n,m}^g h_m(k+p-1|k)) + b_n^g \right)
\end{aligned} \quad (52)$$

and

$$\begin{aligned}
o_n(k+p|k) = & \sigma \left(\sum_{m=1}^{I_{uf}(p)} w_{n,m}^o u(k-m+p|k) + \sum_{m=I_{uf}(p)+1}^{n_B} w_{n,I_{uf}(p)+m}^o u(k-m+p) \right. \\
& + \sum_{m=1}^{I_{yf}(p)} w_{n,n_B+m}^o \hat{y}^{LSTM}(k-m+p|k) + \sum_{m=I_{yf}(p)+1}^{n_A} w_{n,I_{yf}(p)+m}^o y(k-m+p) \\
& \left. + \sum_{m=1}^{n_N} r_{n,m}^o h_m(k+p-1|k) + b_n^o \right)
\end{aligned} \quad (53)$$

Then, the predicted cell and hidden states can be determined from Equations (21) and (22)

$$c_n(k+p|k) = f_n(k+p|k)c_n(k-1+p|k) + i_n(k+p|k)g_n(k+p|k) \quad (54)$$

$$h_n(k+p|k) = o_n(k+p|k) \tanh(c_n(k+p|k)) \quad (55)$$

Let us stress that the above equations have to be used recurrently for $p = 1, \dots, N$. Finally, the predicted output of the i -th LSTM sub-model can be computed from Equation (23)

$$\hat{y}_{(i)}^{LSTM}(k+p|k) = \sum_{n=1}^{n_N} w_y^n \left(h_n(k+p|k) \right) + b_y \quad (56)$$

which can be also expressed as

$$\hat{y}_{(i)}^{\text{LSTM}}(k+p|k) = \sum_{n=1}^{n_N} w_y^n \left(o_n(k+p|k) \tanh(f_n(k+p|k)c_n(k+p-1|k) + i_n(k+p|k)g_n(k+p|k)) \right) + b_y + d(k) \quad (57)$$

3.6. FP Model Prediction

Using Equations (4) and (5), we find model states and the output for the future time instant $k+1$

$$x_1(k+p|k) = f_1(x_1(k+p-1|k), \dots, x_{n_x}(k+p-1|k), u(k+p-1|k)) \quad (58)$$

$$\vdots$$

$$x_{n_x}(k+p|k) = f_{n_x}(x_1(k+p-1|k), \dots, x_{n_x}(k+p-1|k), u(k+p-1|k)) \quad (59)$$

$$y^{\text{FP}}(k+p|k) = g(x_1(k+p-1|k), \dots, x_{n_x}(k+p-1|k)) \quad (60)$$

To simplify the following calculations, let us start with computing the prediction of the states for the time instant $k+p$

$$\hat{x}_1(k+1|k) = f_1(k+1|k) = f_1(x_1(k), \dots, x_{n_x}(k), u(k|k)) + v_1(k) \quad (61)$$

$$\vdots$$

$$\hat{x}_{n_x}(k+1|k) = f_{n_x}(k+1|k) = f_{n_x}(x_1(k), \dots, x_{n_x}(k), u(k|k)) + v_{n_x}(k) \quad (62)$$

From Equation (6), we find the corresponding predicted controlled variable:

$$\hat{y}^{\text{FP}}(k+1|k) = g(k+1|k) + d(k) = g(\hat{x}_1(k+1|k), \dots, \hat{x}_{n_x}(k+1|k)) + d(k) \quad (63)$$

Next, we can determine the predictions for the subsequent sampling instants:

$$\hat{x}_1(k+p|k) = f_1(k+p|k) = f_1(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k)) + v_1(k) \quad (64)$$

$$\vdots$$

$$\hat{x}_{n_x}(k+p|k) = f_{n_x}(k+p|k) = f_{n_x}(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k)) + v_{n_x}(k) \quad (65)$$

$$\hat{y}^{\text{FP}}(k+p|k) = g(k+p|k) + d(k) = g(\hat{x}_1(k+p|k), \dots, \hat{x}_{n_x}(k+p|k)) + d(k) \quad (66)$$

where $p = 2, \dots, N$. The state and output disturbances (prediction errors), respectively, are computed as the measurements compared with the outputs of the corresponding model equations

$$v_1(k) = x_1(k) - f_1(x_1(k-1), \dots, x_{n_x}(k-1), u(k-1)) \quad (67)$$

$$\vdots$$

$$v_{n_x}(k) = x_{n_x}(k) - f_{n_x}(x_1(k-1), \dots, x_{n_x}(k-1), u(k-1)) \quad (68)$$

$$d(k) = y(k) - g(x_1(k-1), \dots, x_{n_x}(k-1)) \quad (69)$$

3.7. PIHNN Model Derivatives

The entries of the matrix $H(k)$ (Equation (37)) are computed from Equation (38). Differentiation of Equation (44) yields

$$\frac{\partial \hat{y}^{\text{PIHNN}}(k+p|k)}{\partial u(k+r|k)} = \frac{\sum_{n=1}^{n_{\text{LSTM}}} \frac{\partial \hat{y}_n^{\text{LSTM}}(k+p|k)}{\partial u(k+r|k)} \mu_n^{\text{LSTM}}(k) + \frac{\partial \hat{y}^{\text{FP}}(k+p|k)}{\partial u(k+r|k)} \left(\sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k) \right)}{\sum_{n=1}^{n_{\text{LSTM}}} \mu_n^{\text{LSTM}}(k) + \sum_{n=1}^{n_{\text{FP}}} \mu_n^{\text{FP}}(k)} \quad (70)$$

Let us note that the derivatives of the whole PIHNN model depend on the LSTM and FP sub-model derivatives.

3.8. LSTM Model Derivatives

Derivatives for LSTM sub-models are calculated by differentiating Equation (57)

$$\frac{\partial \hat{y}_{(i)}^{\text{LSTM}}(k+p|k)}{\partial u(k+r|k)} = \sum_{n=1}^{n_{\text{N}}} w_n^y \frac{\partial h_n(k+p|k)}{\partial u(k+r|k)} \quad (71)$$

For all $p = 1, \dots, N$ and $r = 0, \dots, N_u - 1$, the subsequent step involves the application of the chain rule of differentiation. Initially, it is imperative to determine the derivatives of gates i , f , g , and o . We proceed to differentiate Equation (50):

$$\begin{aligned} \frac{\partial i_n(k+p|k)}{\partial u(k+r|k)} &= i_n(k+p|k)(1-i_n(k+p|k)) \left(\sum_{m=1}^{I_{\text{uf}}(p)} w_{n,m}^i \frac{\partial u(k-m+p|k)}{\partial u(k+r|k)} \right. \\ &\quad \left. + \sum_{m=1}^{I_{\text{yf}}(p)} w_{n,n_{\text{B}}+m}^i \frac{\partial \hat{y}^{\text{LSTM}}(k-m+p|k)}{\partial u(k+r|k)} + \sum_{m=1}^{n_{\text{N}}} r_{n,m}^i \frac{\partial h_m(k+p-1|k)}{\partial u(k+r|k)} \right) \end{aligned} \quad (72)$$

Equation (51) gives

$$\begin{aligned} \frac{\partial f_n(k+p|k)}{\partial u(k+r|k)} &= f_n(k+p|k)(1-f_n(k+p|k)) \left(\sum_{m=1}^{I_{\text{uf}}(p)} w_{n,m}^f \frac{\partial u(k-m+p|k)}{\partial u(k+r|k)} \right. \\ &\quad \left. + \sum_{m=1}^{I_{\text{yf}}(p)} w_{n,n_{\text{B}}+m}^f \frac{\partial \hat{y}^{\text{LSTM}}(k-m+p|k)}{\partial u(k+r|k)} + \sum_{m=1}^{n_{\text{N}}} r_{n,m}^f \frac{\partial h_m(k+p-1|k)}{\partial u(k+r|k)} \right) \end{aligned} \quad (73)$$

from Equation (52), we obtain

$$\begin{aligned} \frac{\partial g_n(k+p|k)}{\partial u(k+r|k)} &= (1-g_n^2(k+p|k)) \left(\sum_{m=1}^{I_{\text{uf}}(p)} w_{n,m}^g \frac{\partial u(k-m+p|k)}{\partial u(k+r|k)} \right. \\ &\quad \left. + \sum_{m=1}^{I_{\text{yf}}(p)} w_{n,n_{\text{B}}+m}^g \frac{\partial \hat{y}^{\text{LSTM}}(k-m+p|k)}{\partial u(k+r|k)} + \sum_{m=1}^{n_{\text{N}}} r_{n,m}^g \frac{\partial h_m(k+p-1|k)}{\partial u(k+r|k)} \right) \end{aligned} \quad (74)$$

Finally, using Equation (53), we derive

$$\begin{aligned} \frac{\partial o_n(k+p|k)}{\partial u(k+r|k)} &= o_n(k+p|k)(1-o_n(k+p|k)) \left(\sum_{m=1}^{I_{\text{uf}}(p)} w_{n,m}^o \frac{\partial u(k-m+p|k)}{\partial u(k+r|k)} \right. \\ &\quad \left. + \sum_{m=1}^{I_{\text{yf}}(p)} w_{n,n_{\text{B}}+m}^o \frac{\partial \hat{y}^{\text{LSTM}}(k-m+p|k)}{\partial u(k+r|k)} + \sum_{m=1}^{n_{\text{N}}} r_{n,m}^o \frac{\partial h_m(k+p-1|k)}{\partial u(k+r|k)} \right) \end{aligned} \quad (75)$$

The following step involves computing the derivative of the cell state c using Equation (54)

$$\begin{aligned} \frac{\partial c_n(k+p|k)}{\partial u(k+r|k)} &= \frac{\partial f_n(k+p|k)}{\partial u(k+r|k)} c_n(k+p-1|k) + f_n(k+p|k) \frac{\partial c_n(k+p-1|k)}{\partial u(k+r|k)} \\ &\quad + \frac{\partial i_n(k+p|k)}{\partial u(k+r|k)} g_n(k+p|k) + i_n(k+p|k) \frac{\partial g_n(k+p|k)}{\partial u(k+r|k)} \end{aligned} \quad (76)$$

and from Equation (54), we can derive the derivatives of the hidden state h

$$\begin{aligned} \frac{\partial h_n(k+p|k)}{\partial u(k+r|k)} &= \frac{\partial o_n(k+p|k)}{\partial u(k+r|k)} \tanh(c_n(k+p|k)) \\ &\quad + o_n(k+p|k) \left(1 - \tanh^2(c_n(k+p|k))\right) \frac{\partial c_n(k+p|k)}{\partial u(k+r|k)} \end{aligned} \quad (77)$$

3.9. FP Model Derivatives

We start by finding derivatives of the predicted state variables for the sampling instant $k+1$. Differentiating Equations (61) and (62), we obtain

$$\begin{aligned} \frac{\partial \hat{x}_1(k+1|k)}{\partial u(k+r|k)} &= \sum_{i=1}^{n_x} \frac{\partial f_1(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial x_i(k)} \frac{\partial x_i(k)}{\partial u(k+r|k)} \\ &\quad + \frac{\partial f_1(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial u(k|k)} \frac{\partial u(k|k)}{\partial u(k+r|k)} \end{aligned} \quad (78)$$

$$\begin{aligned} &\vdots \\ \frac{\partial \hat{x}_{n_x}(k+1|k)}{\partial u(k+r|k)} &= \sum_{i=1}^{n_x} \frac{\partial f_{n_x}(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial x_i(k)} \frac{\partial x_i(k)}{\partial u(k+r|k)} \\ &\quad + \frac{\partial f_{n_x}(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial u(k|k)} \frac{\partial u(k|k)}{\partial u(k+r|k)} \end{aligned} \quad (79)$$

Knowing that

$$\frac{\partial u^{\text{traj}}(k+p|k)}{\partial u^{\text{traj}}(k+r|k)} = \begin{cases} 1 & \text{if } p = r \text{ or } (p > r \text{ and } r = N_u - 1) \\ 0 & \text{otherwise} \end{cases} \quad (80)$$

we can simplify Equations (78) and (79) to

$$\frac{\partial \hat{x}_1(k+1|k)}{\partial u(k+r|k)} = \frac{\partial f_1(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial u(k|k)} \frac{\partial u(k|k)}{\partial u(k+r|k)} \quad (81)$$

$$\begin{aligned} &\vdots \\ \frac{\partial \hat{x}_{n_x}(k+1|k)}{\partial u(k+r|k)} &= \frac{\partial f_{n_x}(x_1(k), \dots, x_{n_x}(k), u(k|k))}{\partial u(k|k)} \frac{\partial u(k|k)}{\partial u(k+r|k)} \end{aligned} \quad (82)$$

The next step is to find the derivative for the FP model states and the controlled variable for prediction at the sampling instant $k+p$, where $p = 2, \dots, N$. From Equation (63), we have

$$\frac{\partial \hat{y}^{\text{FP}}(k+1|k)}{\partial u(k+r|k)} = \sum_{i=1}^{n_x} \frac{\partial g(\hat{x}_1(k+1|k), \dots, \hat{x}_{n_x}(k+1|k))}{\partial \hat{x}_i(k+1|k)} \frac{\partial \hat{x}_i(k+1|k)}{\partial u(k+r|k)} \quad (83)$$

Next, we can determine the derivatives when $p = 2, \dots, N$. We start with the state variables. From Equations (64) and (65), we obtain

$$\begin{aligned} \frac{\partial \hat{x}_1(k+p|k)}{\partial u(k+r|k)} &= \sum_{i=1}^{n_x} \frac{\partial f_1(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k))}{\partial \hat{x}_i(k+p-1|k)} \\ &\quad \times \frac{\partial \hat{x}_i(k+p-1|k)}{\partial u(k+r|k)} \\ &\quad + \frac{\partial f_1(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k))}{\partial u(k+p-1|k)} \\ &\quad \times \frac{\partial u(k+p-1|k)}{\partial u(k+r|k)} \\ &\quad \vdots \end{aligned} \quad (84)$$

$$\begin{aligned}
\frac{\partial \hat{x}_{n_x}(k+p|k)}{\partial u(k+r|k)} &= \sum_{i=1}^{n_x} \frac{\partial f_{n_x}(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k))}{\partial \hat{x}_i(k+p-1|k)} \\
&\quad \times \frac{\partial \hat{x}_i(k+p-1|k)}{\partial u(k+r|k)} \\
&\quad + \frac{\partial f_{n_x}(\hat{x}_1(k+p-1|k), \dots, \hat{x}_{n_x}(k+p-1|k), u(k+p-1|k))}{\partial u(k+p-1|k)} \\
&\quad \times \frac{\partial u(k+p-1|k)}{\partial u(k+r|k)}
\end{aligned} \quad (85)$$

Finally, we can find the predictions of the FP sub-model output using Equation (66)

$$\frac{\partial \hat{y}^{\text{FP}}(k+p|k)}{\partial u(k+r|k)} = \sum_{i=1}^{n_x} \frac{\partial g(\hat{x}_1(k+p|k), \dots, \hat{x}_{n_x}(k+p|k))}{\partial \hat{x}_i(k+p|k)} \frac{\partial \hat{x}_i(k+p|k)}{\partial u(k+r|k)} \quad (86)$$

4. Results

4.1. Polymerization Process Description

The process under study is a polymerization reactor [43] that is frequently used as a benchmark to assess the usefulness of models and control methods, e.g., [20,27]. This process is characterized by a single input, representing the initiator's flow rate, denoted as F_I ($\text{m}^3 \text{h}^{-1}$). Likewise, it has a single output, the number average molecular weight (NAMW) (kg kmol^{-1}). Both input and output signals have been appropriately normalized to facilitate the training of neural networks. The scaling is defined as follows: $u = 100(F_I - \bar{F}_I)$ and $y = 0.0001(\text{NAMW} - \bar{\text{NAMW}})$. The values at the nominal operating point are $\bar{F}_I = 0.016783$ and $\bar{\text{NAMW}} = 20,000$. The polymerization process operates with a sampling time $T = 1.8$ seconds.

Let us note that the predictions determined from the LSTM sub-models are universal, as derived in Section 3.5. Similarly, let us note that the derivatives matrix determined from the LSTM sub-models are universal, as derived in Section 3.8. Hence, it is only necessary to derive specific equations for prediction using the specific first-principle model of the process. Next, we have to derive equations for the derivatives matrix.

4.2. First-Principle Model for Polymerization Process and Its Use in MPC

The continuous-time first-principle model of the polymerization process [43] is discretized using the Euler method. The discrete-time model has the following form:

$$x_1(k) = T \left(60 - 10x_1(k-1) \sqrt{x_2(k-1)} \right) + x_1(k-1) \quad (87)$$

$$x_2(k) = T \left(80u(k-1) - 10.1022x_2(k-1) \right) + x_2(k-1) \quad (88)$$

$$\begin{aligned}
x_3(k) &= T \left(0.0024121x_1(k-1) \sqrt{x_2(k-1)} + 0.112191x_2(k-1) - 10x_3(k-1) \right) \\
&\quad + x_3(k-1)
\end{aligned} \quad (89)$$

$$x_4(k) = T \left(245.978x_1(k-1) \sqrt{x_2(k-1)} - 10x_4(k-1) \right) + x_4(k-1) \quad (90)$$

$$y^{\text{FP}}(k) = \frac{x_4(k)}{x_3(k)} \quad (91)$$

where the model parameters are: $p_1 = 60T$, $p_2 = -10T$, $p_3 = 80T$, $p_4 = -10.1022T + 1$, $p_5 = 0.0024121T$, $p_6 = 0.112191T$, $p_7 = -10T + 1$, $p_8 = 245.978T$, $p_9 = -10T + 1$.

It is important to note that to emulate the imperfections and inaccuracies of the FP model, we introduced a 20 percent increase to the gain of the model during the simulation experiments, i.e.,

$$y_{\text{disturbed}}^{\text{FP}}(k) = 1.2y^{\text{FP}}(k) = 1.2 \frac{x_4(k)}{x_3(k)} \quad (92)$$

For the PIHNN model used in our MPC algorithm, we have to derive equations for the prediction using the specific FP model of the considered benchmark system and the general rules formulated in Section 3.6. They will allow to calculate the predicted trajectory $\hat{y}^{\text{traj}}(k)$, as defined by Equation (34). We start with determining prediction equations when $p = 1$. From Equations (87)–(91), we obtain

$$\hat{x}_1(k+1|k) = p_1 + p_2 x_1(k) \sqrt{x_2(k)} + x_1(k) + v_1(k) \quad (93)$$

$$\hat{x}_2(k+1|k) = p_3 u(k|k) + p_4 x_2(k) + v_2(k) \quad (94)$$

$$\hat{x}_3(k+1|k) = p_5 x_1(k) \sqrt{x_2(k)} + p_6 x_2(k) + p_7 x_3(k) + v_3(k) \quad (95)$$

$$\hat{x}_4(k+1|k) = p_8 x_1(k) \sqrt{x_2(k)} + p_9 x_4(k) + v_4(k) \quad (96)$$

$$\hat{y}^{\text{FP}}(k+1|k) = \frac{\hat{x}_4(k+1|k)}{\hat{x}_3(k+1|k)} + d(k) \quad (97)$$

The state and output disturbances are derived from the general Equations (67)–(69), respectively, which gives

$$v_1(k) = x_1(k) - p_1 + p_2 x_1(k-1) \sqrt{x_2(k-1)} + x_1(k-1) \quad (98)$$

$$v_2(k) = x_2(k) - p_3 u(k-1) + p_4 x_2(k-1) \quad (99)$$

$$v_3(k) = x_3(k) - p_5 x_1(k-1) \sqrt{x_2(k-1)} + p_6 x_2(k-1) + p_7 x_3(k-1) \quad (100)$$

$$v_4(k) = x_4(k) - p_8 x_1(k-1) \sqrt{x_2(k-1)} + p_9 x_4(k-1) \quad (101)$$

$$d(k) = y(k) - \frac{x_4(k)}{x_3(k)} \quad (102)$$

Next, we find the equations for state and output predictions for $p = 2, \dots, N$

$$\hat{x}_1(k+p|k) = p_1 + p_2 \hat{x}_1(k+p-1|k) \sqrt{\hat{x}_2(k+p-1|k)} + \hat{x}_1(k+p-1|k) + v_1(k) \quad (103)$$

$$\hat{x}_2(k+p|k) = p_3 u(k+p-1|k) + p_4 \hat{x}_2(k+p-1|k) + v_2(k) \quad (104)$$

$$\begin{aligned} \hat{x}_3(k+p|k) &= p_5 \hat{x}_1(k+p-1|k) \sqrt{\hat{x}_2(k+p-1|k)} + p_6 \hat{x}_2(k+p-1|k) \\ &\quad + p_7 \hat{x}_3(k+p-1|k) + v_3(k) \end{aligned} \quad (105)$$

$$\hat{x}_4(k+p|k) = p_8 \hat{x}_1(k+p-1|k) \sqrt{\hat{x}_2(k+p-1|k)} + p_9 \hat{x}_4(k+p-1|k) + v_4(k) \quad (106)$$

$$\hat{y}^{\text{FP}}(k+p|k) = \frac{\hat{x}_4(k+p|k)}{\hat{x}_3(k+p|k)} + d(k) \quad (107)$$

Using the above predictions generated by the FP model, we have to determine derivatives of the predicted trajectory of the controlled variable with respect to the trajectory of the manipulated variable, i.e., the derivative matrix $H(k)$, as defined by Equation (38). For this purpose, we use the general rules formulated in Section 3.9. We consider Equations (81) and (82) and we obtain

$$\frac{\partial \hat{x}_1(k+1|k)}{\partial u(k+r|k)} = 0 \quad (108)$$

$$\frac{\partial \hat{x}_2(k+1|k)}{\partial u(k+r|k)} = 80T \frac{\partial u(k|k)}{\partial u(k+r|k)} \quad (109)$$

$$\frac{\partial \hat{x}_3(k+1|k)}{\partial u(k+r|k)} = 0 \quad (110)$$

$$\frac{\partial \hat{x}_4(k+1|k)}{\partial u(k+r|k)} = 0 \quad (111)$$

Equation (83) allows us to express the output derivatives as

$$\frac{\partial \hat{y}^{\text{FP}}(k+p|k)}{\partial u(k+r|k)} = 0 \quad (112)$$

Finally, we use Equations (84)–(86) to determine the state variable and output derivatives, respectively

$$\begin{aligned} \frac{\partial \hat{x}_1(k+p|k)}{\partial u(k+r|k)} &= p_2 \frac{\partial \hat{x}_1(k+p-1|k)}{\partial u(k+r|k)} \sqrt{\hat{x}_2(k+p-1|k) - 0.5p_2\hat{x}_1(k+p-1|k)} \\ &\quad \times (\hat{x}_2(k+p-1|k))^{-2} \frac{\partial \hat{x}_2(k+p-1|k)}{\partial u(k+r|k)} + \frac{\hat{x}_1(k+p-1|k)}{\partial u(k+r|k)} \end{aligned} \quad (113)$$

$$\frac{\partial \hat{x}_2(k+p|k)}{\partial u(k+r|k)} = p_3 \frac{\partial u(k+p-1|k)}{\partial u(k+r|k)} + p_4 \frac{\hat{x}_2(k+p-1|k)}{\partial u(k+r|k)} \quad (114)$$

$$\begin{aligned} \frac{\partial \hat{x}_3(k+p|k)}{\partial u(k+r|k)} &= p_5 \frac{\partial \hat{x}_1(k+p-1|k)}{\partial u(k+r|k)} \sqrt{\hat{x}_2(k+p-1|k) - 0.5p_5\hat{x}_1(k+p-1|k)} \\ &\quad \times (\hat{x}_2(k+p-1|k))^{-2} \frac{\partial \hat{x}_2(k+p-1|k)}{\partial u(k+r|k)} + p_6 \frac{\hat{x}_2(k+p-1|k)}{\partial u(k+r|k)} \\ &\quad + p_7 \frac{\hat{x}_3(k+p-1|k)}{\partial u(k+r|k)} \end{aligned} \quad (115)$$

$$\begin{aligned} \frac{\partial \hat{x}_4(k+p|k)}{\partial u(k+r|k)} &= p_8 \frac{\partial \hat{x}_1(k+p-1|k)}{\partial u(k+r|k)} \sqrt{\hat{x}_2(k+p-1|k) - 0.5p_8\hat{x}_1(k+p-1|k)} \\ &\quad \times (\hat{x}_2(k+p-1|k))^{-2} \frac{\partial \hat{x}_2(k+p-1|k)}{\partial u(k+r|k)} + p_9 \frac{\hat{x}_4(k+p-1|k)}{\partial u(k+r|k)} \end{aligned} \quad (116)$$

$$\frac{\partial \hat{y}^{\text{FP}}(k+p|k)}{\partial u(k+r|k)} = \frac{1}{\hat{x}_3(k+p|k)^2} \left(\frac{\partial \hat{x}_4(k+p|k)}{\partial u(k+r|k)} \hat{x}_3(k+p|k) - \hat{x}_4(k+p|k) \frac{\partial \hat{x}_3(k+p|k)}{\partial u(k+r|k)} \right) \quad (117)$$

for all $p = 2, \dots, N$ and $r = 0, \dots, N_u - 1$.

4.3. LSTM Model for Polymerization Process

Two separate training datasets have been collected from the simulated process (i.e., from simulation of the continuous-time first-principle models), for different operating conditions, as follows:

1. dataset 1 has been collected for the range of the manipulated variable $0.003 < F_1 < 0.0129$, which results in the controlled variable $2.78 \times 10^4 < \text{NAMW} < 4.55 \times 10^4$.
2. dataset 2 has been collected for $0.05 < F_1 < 0.06$ which results in $1.41 \times 10^4 < \text{NAMW} < 1.54 \times 10^4$.

The datasets were then used to train two LSTM models, denoted thereafter as LSTM1 and LSTM2. Both models have been trained with the same parameters:

- the number of neurons $n_N = 7$;
- the order of dynamics $n_A = 0, n_B = 1$.

LSTM models have been trained in MATLAB on a PC equipped with an Nvidia GeForce 970 GTX GPU, an Intel i5-3450 CPU and 16 GB of RAM. We have employed the Adam optimization algorithm with a learning rate of 0.001 and a maximum of 1000 training epochs.

4.4. Modeling Quality of LSTM and FP Models

The modeling quality of all sub-models developed for the polymerization process can be compared in Figure 5. In this comparison, we can see the individual outputs of all sub-models when operating independently with the test dataset. LSTM1, trained predominantly with data featuring large NAMW values, unsurprisingly demonstrates exceptional performance when dealing with such high NAMW values. However, the model's capability to provide correct outputs diminishes when it encounters data not present in the training dataset. Conversely, LSTM2, trained with low NAMW values, excels

when the NAMW values are indeed low. However, it exhibits subpar performance when attempting to model high NAMW values. Notably, in the FP model with the increased gain performs poorly across the entire range of NAMW values.

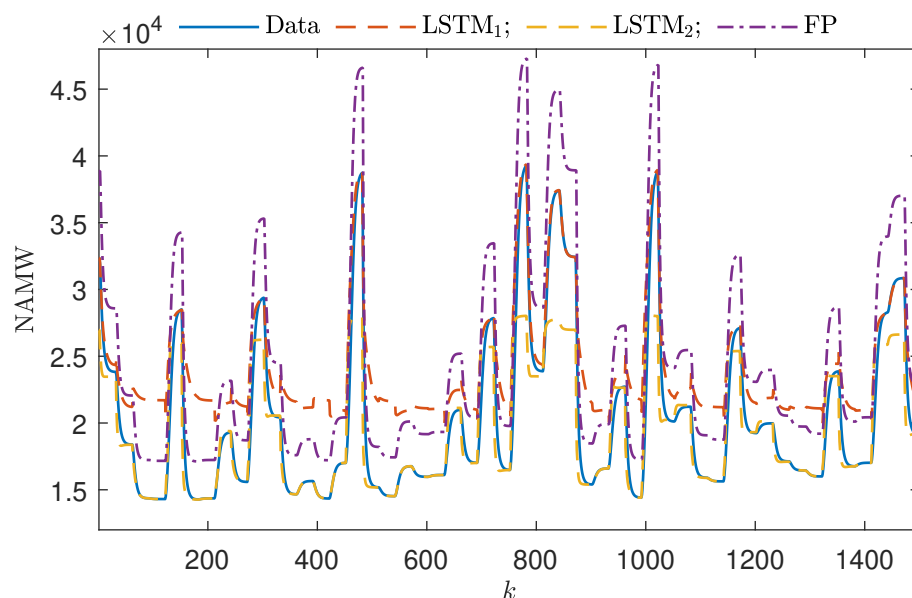


Figure 5. A total of 1000 samples of the validation dataset vs. outputs of two local LSTM sub-models and the FP model with an incorrect gain.

4.5. Development of PIHNN Models

Once all the sub-models have been prepared, the next step to design the PIHNN model is to develop the DF block. Various membership function shapes have been tested, i.e.:

- PIHNN model ver. 1—initial trapezoidal functions;
- PIHNN model ver. 2—optimized trapezoidal functions;
- PIHNN model ver. 3—initial trapezoidal functions;
- PIHNN model ver. 4—optimized trapezoidal functions;
- PIHNN model ver. 5—initial trapezoidal functions;
- PIHNN model ver. 6—optimized trapezoidal functions.

The membership functions are depicted in Figure 6. Our understanding of the sub-models has guided the initial choices of these shapes. The plots display fuzzified variable values along the horizontal axis, specifically representing the NAMW output of the polymerization reactor. Along the vertical axis, one can find the membership function values. Each membership function corresponds to a particular model. LSTM1, which was trained on data with large NAMW values, is most effective when dealing with large NAMW values. The blue membership functions on the plot indicate the range of NAMW values for which prioritizing the use of the LSTM1 model is recommended. LSTM2, characterized by yellow membership functions, is best suited for NAMW values close to the data in its training set, which primarily includes small values of NAMW. In scenarios where NAMW values fall outside the data ranges of both training sets, the most reliable choice is to utilize the FP model, represented by orange membership functions. Once the initial shapes have been determined, the subsequent step involves utilizing an optimization procedure to fine-tune these shapes. The procedure starts with initial membership function shapes, using Levenberg–Marquardt to minimize the overall error of the PIHNN model.

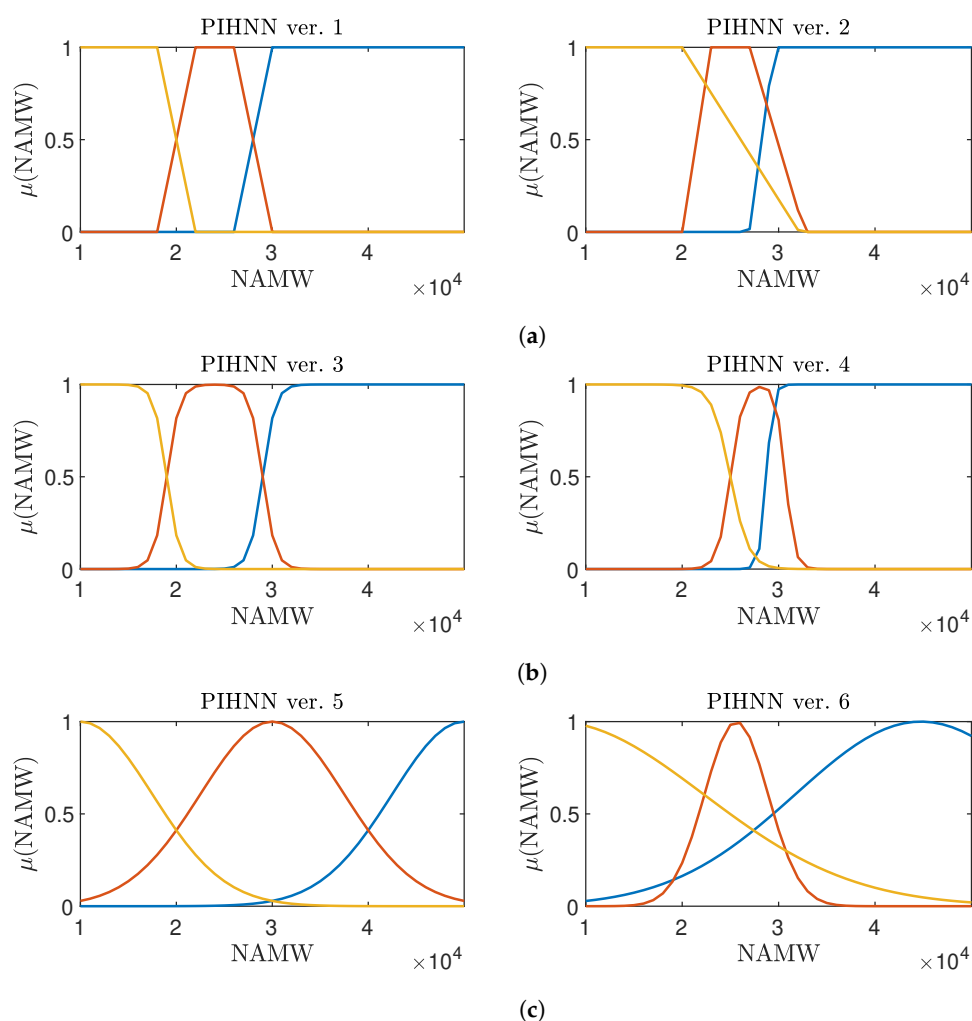


Figure 6. Membership functions for considered fuzzy PIHNN models: fuzzy set 1 (blue), fuzzy set 2 (orange), fuzzy set 3 (yellow). (a) Initial (left) and optimized (right) trapezoidal membership functions; (b) initial (left) and optimized (right) sigmoidal membership functions; (c) initial (left) and optimized (right) Gauss membership functions.

4.6. PIHNN Modeling Quality

The results of the polymerization reactor modeling experiments are presented in Figures 7–9. These figures illustrate the initial 1500 steps of the simulation. Each figure showcases the outputs of two PIHNN models: one with the initial membership function shapes (orange) and the other with optimized (yellow) membership function shapes. These results are compared to the data from the test set. Figure 7 presents the use of the most straightforward decision blocks with trapezoidal membership functions. Even this simplest approach enables the PIHNN model to outperform individual sub-models. The initial shape of the membership function allows the PIHNN structure to represent the data effectively for both small and large values of NAMW. In cases with intermediate values of NAMW, the PIHNN model averages the outputs of the sub-models, while model output still exhibits some deviation from the test data, there is a clear improvement over the FP model. The model with a tuned shape has lower error overall; however, it tends to have poorer modeling quality for both large and small values of NAMW in comparison to the LSTM sub-models.

Figure 8 illustrates the utilization of sigmoidal membership functions in the DF block of the PIHNN model. Here, the sigmoidal shape allows the PIHNN to excel in modeling small, large, and intermediate NAMW values. Importantly, the tendency to average out intermediate NAMW values, as previously observed with the trapezoidal DF model, has

been eliminated with the sigmoidal DF PIHNN model. Adopting sigmoidal functions has resulted in highly accurate modeling of medium NAMW values. When comparing the output signals of the models with the initial and tuned shapes of the membership functions, they exhibit minimal differences, with only slight variations noticeable for intermediate values of NAMW.

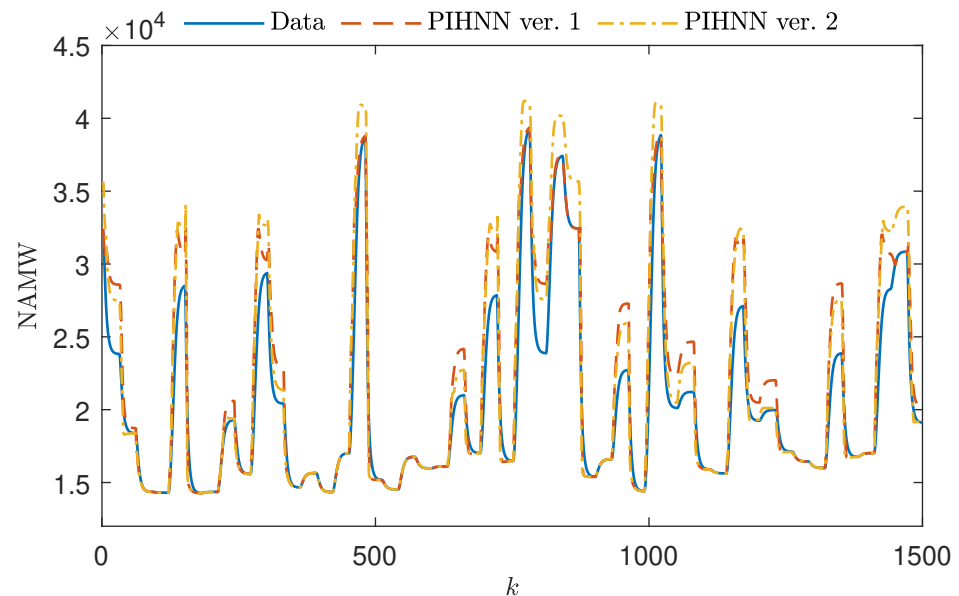


Figure 7. A total of 1000 samples of the validation dataset vs. the output of initial and optimized fuzzy PIHNN structures with trapezoidal MFs (PIHNN models ver. 1 and ver. 2).

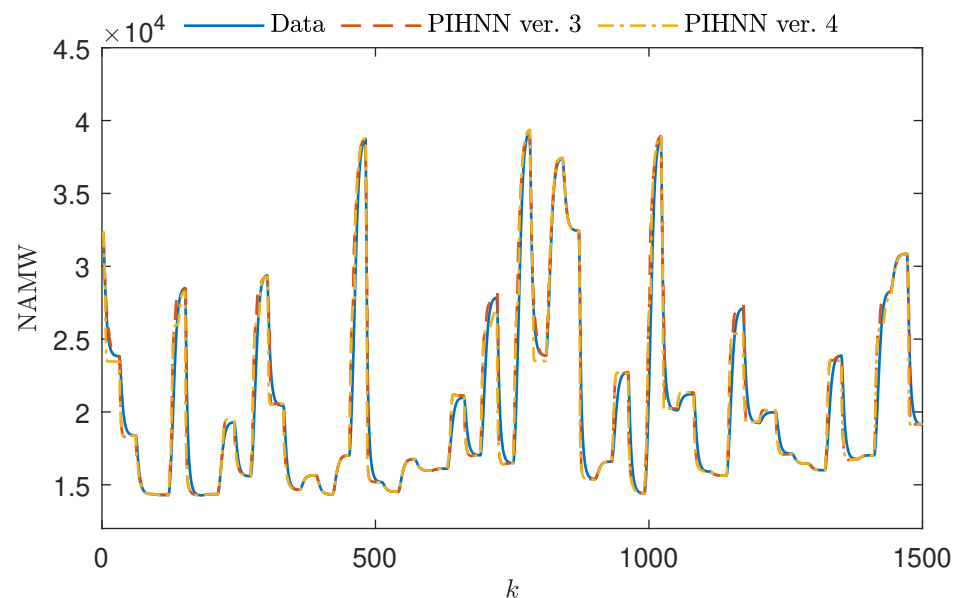


Figure 8. A total of 1000 samples of the validation dataset vs. the output of initial and optimized fuzzy PIHNN structures with sigmoid MFs (PIHNN models ver. 3 and ver. 4).

Finally, Figure 9 presents the utilization of Gaussian membership functions in a DF block of PIHNN model. Here, one can observe that the Gaussian decision model tends to average the values of the three sub-models across the entire spectrum of NAMW variability. This effect is particularly evident in the model with the initial shape of the membership function, where, for large values of NAMW, the model noticeably diverges from the data. As a result, for large NAMW values, PIHNN gives worse results than the independent LSTM1 submodel. Low and intermediate NAMW values are subject to much lower modeling

errors. Although optimizing the shape mitigated this averaging effect somewhat, the model's output still exhibits relatively large errors.

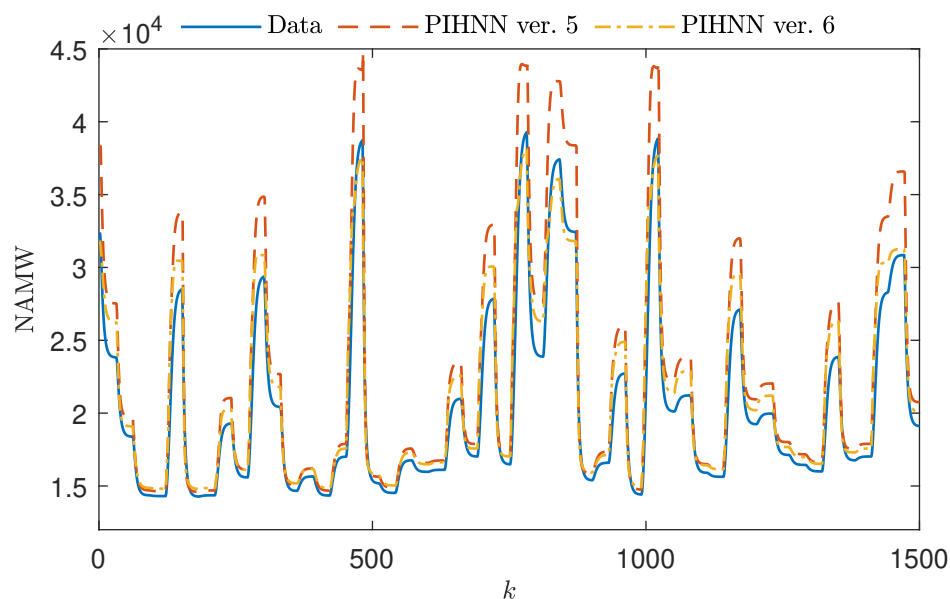


Figure 9. A total of 1000 samples of the validation dataset vs. the output of initial and optimized fuzzy PIHNN structures with Gauss MFs (PIHNN models ver. 5 and ver. 6).

4.7. Validation of MPC Algorithms Using PIHNN Models

The PIHNN model, in six different versions, has been implemented in MPC algorithms. We compare the results obtained from two types of controllers: one with nonlinear optimization (MPC-NO) and the second one recommended in this work, involving linearization around the prediction trajectory (MPC-NPLPT). Table 1 compares the control errors determined for these controllers. First, it is worth noting that the best control quality is achieved for models utilizing DF with Gaussian function shapes. Models employing trapezoidal functions exhibited slightly higher errors, while the poorest performance was observed in models with sigmoidal-shaped functions. This observation may seem counter-intuitive, considering that models with sigmoidal membership functions have smaller modeling errors compared to models with Gaussian ones. It is important to stress that the shape of the closed-loop output trajectory with the MPC controller is affected not only by the quality of the model used but also by the feedback mechanism. Even though Gaussian models exhibit a higher error rate, their inherent averaging characteristic enhances the performance of the MPC controller when coupled with feedback.

Table 1. Control errors of MPC algorithms with different PIHNN models.

Model Type	MPC-NO	MPC-NPLPT
PIHNN ver. 1	3.051	3.116
PIHNN ver. 2	3.031	3.095
PIHNN ver. 3	3.205	3.263
PIHNN ver. 4	3.220	3.290
PIHNN ver. 5	2.935	2.741
PIHNN ver. 6	2.965	3.020

Secondly, Table 1 demonstrates that the MPC-NPLPT controller generally yields slightly higher error values than the MPC-NO one when utilizing the same PIHNN model for prediction. This result is not surprising, as MPC-NPLPT employs a linearized model. During linearization, some of the information present in the nonlinear model is simplified or lost. The exception here is PIHNN model ver. 5, where MPC-NPLPT algorithm provides

better controller performance. This may be attributed to chance where the simplifications happened to benefit the controller's performance in this specific case. However, it is worth noting that the error differences between MPC-NO and MPC-NPLT controllers are minimal for each type of PIHNN model, and both types of controllers work very well.

Table 2 compares the average time required by each MPC controller for control calculations. The computations have been conducted on a PC, and since it is not a real-time system, results may vary on different PCs. Therefore, the results are presented as percentages. The longest time recorded for MPC-NO with PIHNN model ver. 3, which amounted to 140 ms, is considered as 100%. The table reveals that the implementation of the online linearization-based MPC controller significantly reduced the calculation time required, resulting in a 4–5 times decrease compared to nonlinear controllers.

Table 2. Average execution time of MPC algorithms with different PIHNN models.

Model Type	MPC-NO	MPC-NPLT
PIHNN ver. 1	94.4%	22.4%
PIHNN ver. 2	97.2%	23.1%
PIHNN ver. 3	100.0%	21.0%
PIHNN ver. 4	97.2%	23.1%
PIHNN ver. 5	89.3%	24.5%
PIHNN ver. 6	93.7%	23.1%

The results are also visually presented. In Figure 10, one can observe the performance of the MPC algorithm with a DF employing trapezoidal functions. The output signals for the PIHNN model with the initial function shape are swift without overshoot for both low and high values of NAMW. However, for intermediate NAMW values, there is a slightly larger overshoot, and the settling time is extended. The signals are quite similar in the case of DF with a tuned function shape, but there is a greater overshoot for intermediate NAMW values. Additionally, it is worth noting that the results obtained for the MPC-NO controller are practically indistinguishable from those for the MPC-NPLT one.

Figure 11 illustrates the results for sigmoidal membership functions. Here, we can observe that the overshoot becomes more pronounced, particularly for intermediate NAMW values, especially when considering the set-point $NAMW^{sp} = 2.5 \times 10^4$.

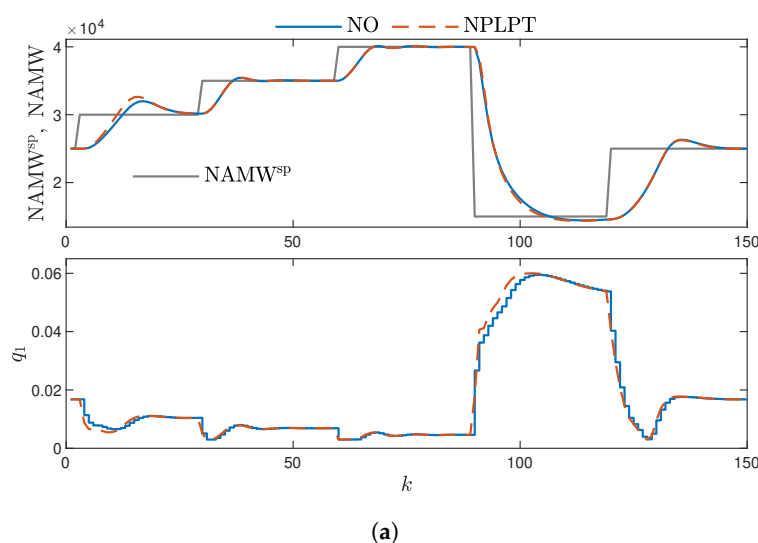


Figure 10. Cont.

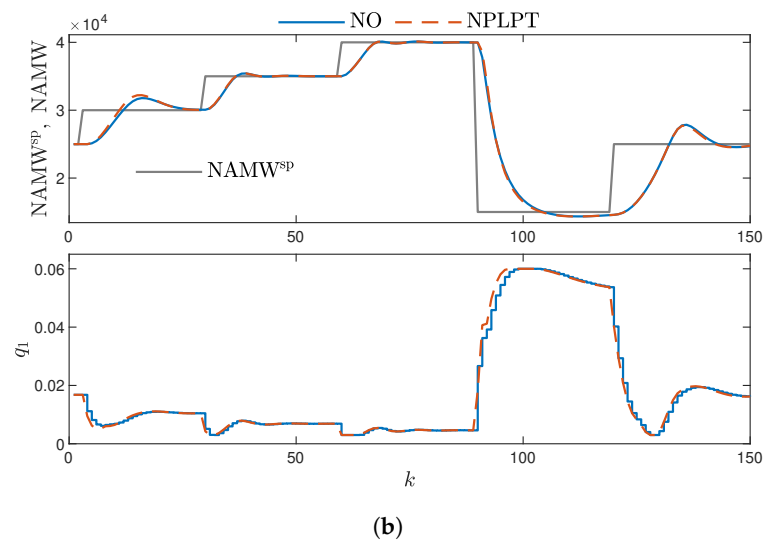


Figure 10. MPC with trapezoidal membership function shapes. (a) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 1; (b) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 2.

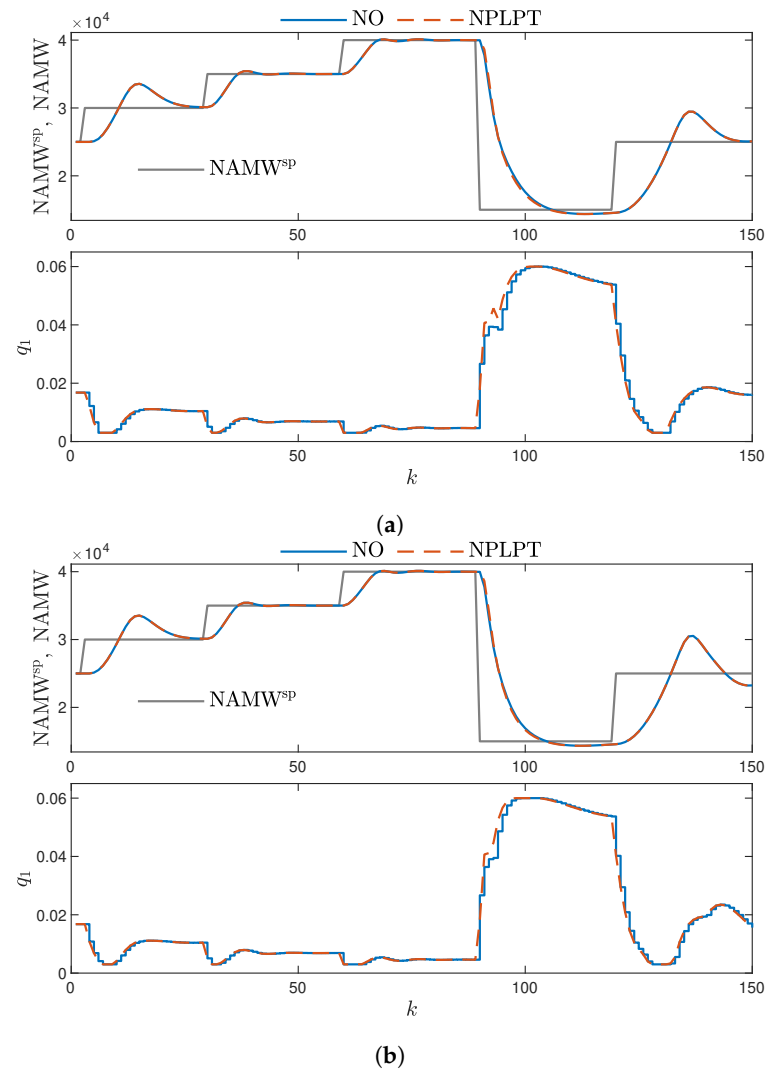


Figure 11. MPC controllers with sigmoidal membership function shapes. (a) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 3; (b) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 4.

The final Figure 12 displays the results of applying Gaussian membership functions. These results are characterized by the shortest settling time and the smallest overshoot. Notably, the controller exhibits excellent performance for average values of NAMW. This observation leads to the conclusion that the averaging nature of Gaussian functions, as seen earlier in the modeling phase (Figure 9), positively impacts the controller's performance when using the model in the MPC scheme. For NAMW values within the range of 2×10^4 to 3×10^4 , the FP model significantly impacts PIHNN performance. As mentioned, the FP model is imperfect, featuring an increased gain of 20%.

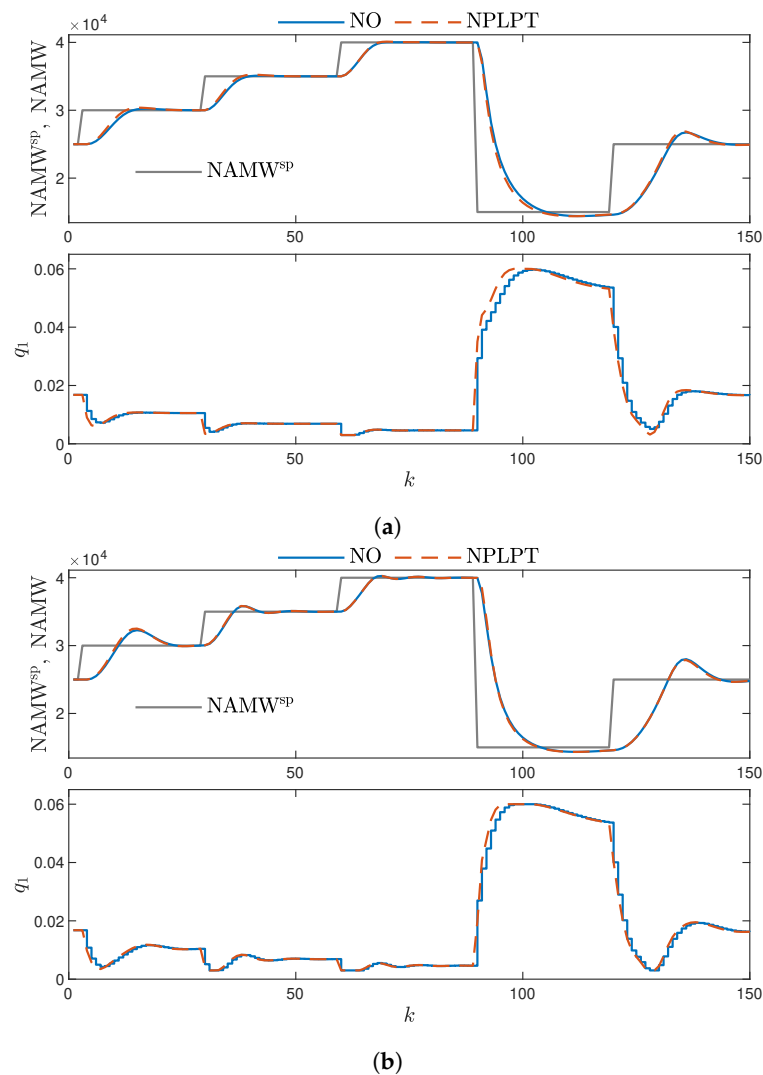


Figure 12. MPC controllers with Gaussian membership function shapes. (a) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 5; (b) MPC-NO and MPC-NPLPT controllers using PIHNN model ver. 6.

5. Conclusions

This work defines a new PIHNN model structure that combines the first-principle process description and data-driven neural sub-models using a specialized data fusion block that relies on fuzzy logic. We consider a very practical case when the available first-principle model is imperfect and the data cannot be measured in the complete range of process operation. By combining an imperfect physical model with data obtained from an incomplete range of operations, we have developed a hybrid model that significantly improves performance across the entire range of signal variability. Secondly, this work develops a computationally efficient MPC controller for the PIHNN model. We show the

efficacy of the PIHNN model and the resulting MPC controller for a simulated polymerization benchmark. We study the efficiency of different data fusion fuzzy blocks and their impact on model accuracy. We recommend tuning, i.e., optimizing the fuzzy membership functions, greatly improving model accuracy. Finally, we show that the described MPC controller based on the PIHNN model gives excellent results. Namely, the obtained control quality is very similar to that possible in MPC relying on nonlinear optimization while its calculation time is a few times shorter. In our future work, we plan to develop a methodology for designing PIHNN structures tailored to processes with multiple inputs and outputs. Additionally, it is interesting to check the impact of employing various decision model types within the data fusion block on PIHNN modeling quality.

Author Contributions: Conceptualisation K.Z. and M.L.; methodology, K.Z. and M.L.; software, K.Z. and M.L.; validation, K.Z. and M.L.; formal analysis, K.Z. and M.L.; investigation, K.Z.; writing—original draft preparation, K.Z. and M.L.; writing—review and editing, K.Z. and M.L.; visualization, K.Z.; supervision, M.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was financed by the Warsaw University of Technology in the framework of the project for the scientific discipline automatic control, electronics and electrical engineering.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: On request from the authors.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Camacho, E.F.; Bordons, C. *Model Predictive Control*; Springer: London, UK, 1999.
2. Tatjewski, P. *Advanced Control of Industrial Processes, Structures and Algorithms*; Springer: London, UK, 2007.
3. Hosen, M.A.; Hussain, M.A.; Mjalli, F.S. Control of polystyrene batch reactors using neural network based model predictive control (NNMPC): An experimental investigation. *Control. Eng. Pract.* **2011**, *19*, 454–467. [\[CrossRef\]](#)
4. Wang, B.; Shahzad, M.; Zhu, X.; Rehman, K.U.; Uddin, S. A Non-linear Model Predictive Control Based on Grey-Wolf Optimization Using Least-Square Support Vector Machine for Product Concentration Control in L-Lysine Fermentation. *Sensors* **2020**, *20*, 3335. [\[CrossRef\]](#)
5. Assandri, A.D.; de Prada, C.; Rueda, A.; Martínez, J.S. Nonlinear parametric predictive temperature control of a distillation column. *Control. Eng. Pract.* **2013**, *21*, 1795–1806. [\[CrossRef\]](#)
6. Carli, R.; Cavone, G.; Ben Othman, S.; Dotoli, M. IoT Based Architecture for Model Predictive Control of HVAC Systems in Smart Buildings. *Sensors* **2020**, *20*, 781. [\[CrossRef\]](#)
7. Alexis, K.; Nikolakopoulos, G.; Tzes, A. Switching model predictive attitude control for a quadrotor helicopter subject to atmospheric disturbances. *ISA Trans.* **2011**, *19*, 1195–1207. [\[CrossRef\]](#)
8. Gruber, J.K.; Doll, M.; Bordons, C. Design and experimental validation of a constrained MPC for the air feed of a fuel cell. *Control. Eng. Pract.* **2009**, *17*, 874–885. [\[CrossRef\]](#)
9. Lima, P.F.; Pereira, G.C.; Mårtensson, J.; Wahlberg, B. Experimental validation of model predictive control stability for autonomous driving. *Control. Eng. Pract.* **2018**, *81*, 244–255. [\[CrossRef\]](#)
10. Yao, F.; Yang, C.; Liu, X.; Zhang, M. Experimental Evaluation on Depth Control Using Improved Model Predictive Control for Autonomous Underwater Vehicle (AUVs). *Sensors* **2018**, *18*, 2321. [\[CrossRef\]](#)
11. Ding, Z.; Sun, C.; Zhou, M.; Liu, Z.; Wu, C. Intersection Vehicle Turning Control for Fully Autonomous Driving Scenarios. *Sensors* **2021**, *21*, 3995. [\[CrossRef\]](#)
12. Bassolillo, S.R.; D’Amato, E.; Notaro, I.; Blasi, L.; Mattei, M. Decentralized Mesh-Based Model Predictive Control for Swarms of UAVs. *Sensors* **2020**, *20*, 4324. [\[CrossRef\]](#)
13. Xiong, L.; Fu, Z.; Zeng, D.; Leng, B. An Optimized Trajectory Planner and Motion Controller Framework for Autonomous Driving in Unstructured Environments. *Sensors* **2021**, *21*, 4409. [\[CrossRef\]](#)
14. Simon, D. *Optimal State Estimation: Kalman, H, and Nonlinear Approaches*; John Wiley and Sons: Hoboken, NJ, USA, 2006.
15. Karimshoushtari, M.; Novara, C.; Tango, F. How Imitation Learning and Human Factors Can Be Combined in a Model Predictive Control Algorithm for Adaptive Motion Planning and Control. *Sensors* **2021**, *21*, 4012. [\[CrossRef\]](#)
16. Miller, A.; Rybczak, M.; Rak, A. Towards the Autonomy: Control Systems for the Ship in Confined and Open Waters. *Sensors* **2021**, *21*, 2286. [\[CrossRef\]](#)
17. Bacci di Capaci, R.; Vaccari, M.; Pannocchia, G. Model predictive control design for multivariable processes in the presence of valve stiction. *J. Process. Control.* **2018**, *71*, 25–34. [\[CrossRef\]](#)

18. Ławryńczuk, M. Modelling and predictive control of a neutralisation reactor using sparse Support Vector Machine Wiener models. *Neurocomputing* **2016**, *205*, 311–328. [\[CrossRef\]](#)
19. Ławryńczuk, M. *Nonlinear Predictive Control Using Wiener Models: Computationally Efficient Approaches for Polynomial and Neural Structures*; Studies in Systems, Decision and Control; Springer: Cham, Switzerland, 2022; Volume 389.
20. Ławryńczuk, M. *Computationally Efficient Model Predictive Control Algorithms: A Neural Network Approach*; Studies in Systems, Decision and Control; Springer: Cham, Switzerland, 2014; Volume 3.
21. Balla, K.M.; Nørgaard, J.T.; Bendtsen, J.D.; Kallesøe, C.S. Model Predictive Control using linearized Radial Basis Function Neural Models for Water Distribution Networks. In Proceedings of the 2019 IEEE Conference on Control Technology and Applications (CCTA), Hong Kong, China, 19–21 August 2019; pp. 368–373.
22. Schwedersky, B.B.; Flesch, R.C.C.; Dangui, H.A.S. Practical nonlinear model predictive control algorithm for Long Short-Term Memory networks. *IFAC-PapersOnLine* **2019**, *52*, 468–473. [\[CrossRef\]](#)
23. Zarzycki, K.; Ławryńczuk, M. Advanced predictive control for GRU and LSTM networks. *Inf. Sci.* **2022**, *616*, 229–254. [\[CrossRef\]](#)
24. Wang, Y. A new concept using LSTM Neural Networks for dynamic system identification. In Proceedings of the 2017 American Control Conference (ACC), Seattle, WA, USA, 24–26 May 2017; pp. 5324–5329.
25. Jordan, I.D.; Sokół, P.A.; Park, I.M. Gated Recurrent Units Viewed Through the Lens of Continuous Time Dynamical Systems. *Front. Comput. Neurosci.* **2021**, *15*, 678158. [\[CrossRef\]](#)
26. Bonassi, F.; da Silva, C.F.O.; Scattolini, R. Nonlinear MPC for Offset-Free Tracking of systems learned by GRU Neural Networks. *IFAC-PapersOnLine* **2021**, *54*, 54–59. [\[CrossRef\]](#)
27. Zarzycki, K.; Ławryńczuk, M. LSTM and GRU Neural Networks as Models of Dynamical Processes Used in Predictive Control: A Comparison for Two Chemical Reactors. *Sensors* **2021**, *21*, 5625. [\[CrossRef\]](#)
28. Li Ping, Z.; Min, X.; Hui-Nan, W. Hybrid control of bifurcation in a predator-prey system with three delays. *Acta Phys. Sin.* **2011**, *60*, 010506. [\[CrossRef\]](#)
29. Lu, L.; Huang, C.; Song, X. Bifurcation control of a fractional-order PD control strategy for a delayed fractional-order prey–predator system. *Eur. Phys. J. Plus* **2023**, *138*, 77. [\[CrossRef\]](#)
30. Xu, C.; Cui, X.; Li, P.; Yan, J.; Yao, L. Exploration on dynamics in a discrete predator–prey competitive model involving feedback controls. *J. Biol. Dyn.* **2023**, *17*, 2220349. [\[CrossRef\]](#)
31. Alhajeri, M.S.; Luo, J.; Wu, Z.; Albalawi, F.; Christofides, P.D. Process structure-based recurrent neural network modeling for predictive control: A comparative study. *Chem. Eng. Res. Des.* **2022**, *179*, 77–89. [\[CrossRef\]](#)
32. Roehrl, M.A.; Runkler, T.A.; Brandtstetter, V.; Tokic, M.; Obermayer, S. Modeling System Dynamics with Physics-Informed Neural Networks Based on Lagrangian Mechanics. *IFAC-PapersOnLine* **2020**, *53*, 9195–9200. [\[CrossRef\]](#)
33. Yang, L.; Meng, X.; Karniadakis, G.E. B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. *J. Comput. Phys.* **2021**, *425*, 109913. [\[CrossRef\]](#)
34. Nascimento, R.G.; Fricke, K.; Viana, F.A. A tutorial on solving ordinary differential equations using Python and hybrid physics-informed neural network. *Eng. Appl. Artif. Intell.* **2020**, *96*, 103996. [\[CrossRef\]](#)
35. Antonelo, E.A.; Camponogara, E.; Seman, L.O.; de Souza, E.R.; Jordanou, J.P.; Hübner, J.F. Physics-Informed Neural Nets-based Control. *arXiv* **2021**, arXiv:2104.02556.
36. Bolderman, M.; Lazar, M.; Butler, H. Physics-Guided Neural Networks for Inversion-based Feedforward Control applied to Linear Motors. In Proceedings of the 2021 IEEE Conference on Control Technology and Applications (CCTA), San Diego, CA, USA, 9–11 August 2021; pp. 1115–1120.
37. Wang, R.; Yu, R. Physics-Guided Deep Learning for Dynamical Systems: A Survey. *arXiv* **2021**, arXiv:2107.01272.
38. Nascimento, R.G.; Corbetta, M.; Kulkarni, C.S.; Viana, F.A. Hybrid physics-informed neural networks for lithium-ion battery modeling and prognosis. *J. Power Sources* **2021**, *513*, 230526.
39. Shi, R.; Mo, Z.; Di, X. Physics-Informed Deep Learning for Traffic State Estimation: A Hybrid Paradigm Informed By Second-Order Traffic Models. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtual, 2–9 February 2021; Volume 35, pp. 540–547.
40. Daw, A.; Karpatne, A.; Watkins, W.; Read, J.; Kumar, V. Physics-guided Neural Networks (PGNN): An Application in Lake Temperature Modeling. *arXiv* **2017**, arXiv:1710.11431.
41. Zarzycki, K.; Ławryńczuk, M. *Physics-Informed Hybrid Neural Network Model for MPC: A Fuzzy Approach*. *Lecture Notes in Networks and Systems*; Pawełczyk M., Bismor D., Ogonowski S., Kacprzyk J., Eds.; Springer Nature Switzerland: Cham, Switzerland, 2023; Volume 708, pp. 183–192.
42. Hochreiter, S. Untersuchungen zu dynamischen neuronalen Netzen. Master's Thesis, Technical University Munich, Munich, Germany, 1991.
43. Doyle, F.J.; Ogunnaike, B.A.; Pearson, R. Nonlinear model-based control using second-order Volterra models. *Automatica* **1995**, *31*, 697–714. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.